FIELD GUIDE · OCTOBER 2026

JUNIT 6 · MOCKITO 5 · ASSERTJ 3 · JAVA 21

```
$ mvn test
└─ JUnit Jupiter ✔
   ├─ OrderServiceTest ✔
   │  ├─ chargesDiscountedTotalSavesAndEmails() ✔
   │  └─ declinedPaymentSavesNothingAndSendsNoEmail() ✔
   └─ DiscountPolicyTest ✔
      └─ ₹1000 → 5% off ✔
[        44 tests successful      ]
[         0 tests failed          ]
```

*The complete reference to*

# Testing *Java* applications

Unit, integration, API, end-to-end, performance and security testing — with JUnit 6, Mockito and AssertJ. Every code example in this guide was compiled and run, and every output shown is real.

`static` `unit` `integration` `contract` `end-to-end` `performance` `security`

`dummy` `stub` `spy` `mock` `fake`

| 6 | 44 | 5 | 1 |
|---|---|---|---|
| chapters | tests, all run | test doubles | cheat sheet |

INSIDE

# Contents


## 01 Foundations

Why we test & the kinds of tests 03
Pyramid, trophy & what makes a good test 04
What to test — and what to leave out 05
Designing test cases 06

## 02 Unit testing with JUnit 6

Setup, architecture & JUnit 6 07
Your first test & the lifecycle 08
Assertions & AssertJ 09
Parameterized, nested & more 10
Writing your own extensions 11
Testing logs and metrics 12

## 03 Test doubles & Mockito

The five test doubles 13
Mockito in practice 14
Failure paths, call order & the API 15
Spies, statics, errors & pitfalls 16
Legacy code: characterization tests & seams 17

## 04 Integration & API testing

Testing a real HTTP API 18
Spring Boot, Testcontainers & contracts 19
@Transactional in tests: what rollback hides 20
Database migrations on real data 21
Messaging & async with Kafka 26
E-mail with GreenMail 27
AWS and Azure emulators 28
Secured APIs: mock auth server 29
Other services: WireMock & retries 30
Contract testing with Pact 31
Time, randomness, threads & flaky tests 32

## 05 Beyond correctness

End-to-end & UI testing 34
Performance testing 35
Security & resilience testing 36

## 06 Quality & process

Test-driven development 38
Coverage & mutation testing 39
Mutation testing with PIT: a worked run 40
PIT in your build and CI 41
Coverage for unit and integration tests 42
Finding duplicate & redundant tests 43
Build plugins for test quality 44
Architecture tests with ArchUnit 45
Parallel tests: threads vs JVMs 46
Singleton containers per JVM 47
Faster builds: cache & shards 48
Build tools & CI pipelines 49
Team testing conventions 50
Check your understanding 51

**Cheat sheet** — one page to pin above your desk 52
**Testing checklist, references & how this guide was verified** 53
**Index A–Z** 54


Versions and facts as of October 2026. Every entry, page reference and footer link is clickable.

### COLOUR LANGUAGE

Each kind of test keeps one colour on every page. Test doubles have their own: `stub` `mock`

| | |
|---|---|
| `static` | checked without running |
| `unit` | one class, in memory |
| `integration` | real parts together |
| `contract` | agreed API shape |
| `end-to-end` | the whole system |
| `performance` | speed under load |
| `security` | attacks & weaknesses |

### CALLOUTS

**TIP**
Practical advice you can apply today.

**WATCH OUT**
A mistake that shows up in real code bases.

**RULE OF THUMB**
A default that is right most of the time.

> *"Program testing can be used to show the presence of bugs, but never to show their absence!"*
>
> — Edsger W. Dijkstra

CHAPTER ONE

# 01 Foundations

A test is code that runs your code and checks the answer. Good tests let you change software quickly without breaking it; bad ones slow you down and still let bugs through. This chapter covers what to test, how much of each kind, and how to pick the cases.

## What tests buy you

### Fast feedback

Find a mistake seconds after you make it, not weeks later in production.

### Safe change

Refactor or upgrade a library and know at once whether behaviour changed.

### Living docs

A test named `declinedPaymentSavesNothing` states a rule that can't go out of date.

### Better design

Testing a class in isolation needs its dependencies passed in, which keeps coupling visible.

## The kinds of tests

| KIND | WHAT IT CHECKS | SCOPE | SPEED | JAVA TOOLS |
|---|---|---|---|---|
| `static` | Code style, likely bugs, types — without running anything | Whole code base | Seconds | javac -Xlint, Error Prone, SpotBugs, Checkstyle · p.44, p.45 |
| `unit` | One class or function; collaborators replaced by doubles | Single process, no I/O | Milliseconds | JUnit, AssertJ, Mockito · p.08, p.14 |
| `integration` | Your code with real neighbours: DB, HTTP, queue, framework | Process + real dependencies | Sub-second to seconds | Spring Boot test slices, Testcontainers, WireMock · p.19, p.30 |
| `contract` | Consumer and provider agree on the API shape | Two services, one at a time | Seconds | Pact, Spring Cloud Contract · p.31 |
| `end-to-end` | A user journey through the deployed system | Browser / client → all services | Seconds to minutes | Playwright, Selenium, REST Assured · p.34 |
| `performance` | Latency, throughput and limits under load | Production-like environment | Minutes to hours | Gatling, JMeter, k6, JMH · p.35 |
| `security` | Known weaknesses, vulnerable libraries, attack paths | Code, dependencies, running app | Minutes | ZAP, OWASP Dependency-Check, Jazzer · p.36, p.29 |

GOOD TO KNOW

Google's *Software Engineering at Google* sorts tests by **size** instead of name: **small** tests run in one process, **medium** tests on one machine (they may call localhost), **large** tests anywhere. The book's reason: "the most important qualities we want from our test suite are speed and determinism", and "small tests, regardless of the scope, are almost always faster and more deterministic".

## Vocabulary

**SUT** System under test — the class or system being tested.

**Test double** A stand-in for a real collaborator.

**Test oracle** How you know the expected answer.

**Coverage** Which code the tests executed.

**Fixture** Everything set up so a test can run.

**Regression** Something that used to work and broke.

**Smoke test** A quick check that the build basically works.

**Mutation score** Share of injected bugs the tests caught.

**Assertion** A check that fails the test if it is false.

**Flaky test** Passes and fails without any code change.

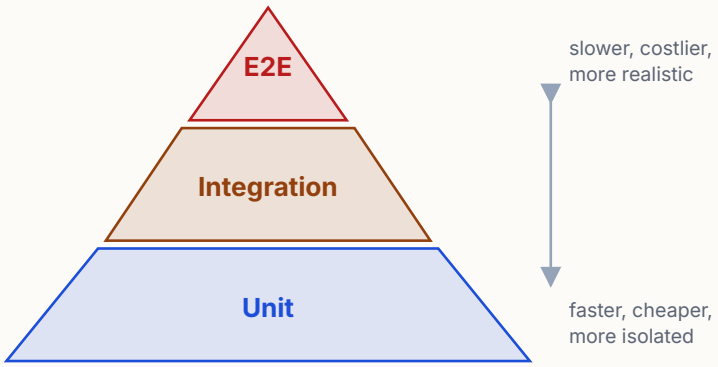
**Acceptance test** Checks a requirement from the user's view.

**Test suite** A group of tests run together.

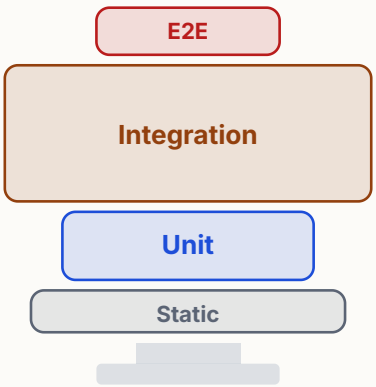
01 · FOUNDATIONS

# Pyramid, trophy & what makes a good test

TEST PYRAMID · COHN, 2009




TESTING TROPHY · DODDS, 2018




**The pyramid** says: write many fast unit tests, fewer integration tests and only a handful of end-to-end tests, because cost and flakiness grow as you go up. Mike Cohn described it in *Succeeding with Agile* (2009).

**The trophy** (Kent C. Dodds, first drawn in a 2018 tweet) puts the most weight on integration tests, with static analysis as the base. Its premise: "The more your tests resemble the way your software is used, the more confidence they can give you."

## FIRST: five properties of a good unit test

| F | I | R | S | T |
|---|---|---|---|---|
| **Fast** | **Independent** | **Repeatable** | **Self-validating** | **Timely** |
| Milliseconds. Slow tests stop being run. | No shared state; any order, in parallel. | Same result anywhere: no real clock, network or randomness. | Pass or fail by itself; nobody reads logs. | Written with (or before) the code, not months later. |

Mnemonic devised by Brett Schuchert and Tim Ottinger; listed in chapter 9 of Robert C. Martin's *Clean Code*.

## Arrange · Act · Assert

MoneyTest.java ✔ passes

```
@Test
@DisplayName("10% off ₹999.99 rounds half-up to ₹899.99")
void percentOffRoundsHalfUp() {
  Money price = Money.of("999.99");  // Arrange

  Money discounted = price.percentOff(10);  // Act

  assertEquals(Money.of("899.99"), discounted); // Assert
}
```

Three blocks, separated by blank lines. **Given / When / Then** is the same idea in BDD words. If "Act" needs more than one line, the API under test is probably awkward.

## Naming tests

| STYLE | EXAMPLE |
|---|---|
| **Behaviour sentence** | `declinedPaymentSavesNothingAndSendsNoEmail` |
| **method_state_result** | `placeOrder_emptyItems_throws` |
| **should…** | `shouldRejectNegativeAmounts` |
| **@DisplayName** | `"10% off ₹999.99 rounds half-up"` |

**DO**

- ✓ Test **behaviour** through the public API
- ✓ One reason to fail per test
- ✓ Make failures explain themselves

**DON'T**

- × Assert on private fields or call order you don't care about
- × Share mutable state between tests
- × Put `if` or loops with logic in a test

01 · FOUNDATIONS

# What to test — and what to leave out

Testing everything equally is the most expensive way to get mediocre safety. Spend tests where a bug would hurt most, write each one so it is worth keeping, and pick the cheapest kind of test that can catch the bug you are worried about.

## Four properties of a valuable test

**Catches regressions**
Fails when behaviour breaks. More real code exercised, more bugs caught.

**Survives refactoring**
Stays green when behaviour *doesn't* change. Tests tied to internals fail for nothing — and get ignored.

**Fast feedback**
Runs in milliseconds, so it runs on every change.

**Cheap to keep**
Easy to read, small setup, one reason to fail.

From Vladimir Khorikov, *Unit Testing: Principles, Practices, and Patterns* (2020). Khorikov's names: protection against regressions, resistance to refactoring, fast feedback, maintainability; he argues the first three can't all be maximised at once. To keep a test resistant to refactoring, **assert on outcomes** (return values, saved state, messages sent), not on which private steps produced them.

## Test by risk

| CODE | RISK | HOW MUCH TESTING |
|---|---|---|
| **Money, permissions, personal data** | high | unit tests for every rule and boundary, mutation testing, an integration test per path, a contract per consumer |
| **Core business rules** | high | unit tests with boundaries and decision tables; PIT on the module |
| **Wiring: SQL, HTTP, messaging, config** | medium | integration tests with the real thing in a container |
| **User journeys that earn money** | medium | a few end-to-end tests: sign-up, checkout, payment |
| **Glue code that only delegates** | low | covered by the integration test above it — no unit test of its own |
| **Getters, generated code, the framework itself** | none | don't test; it is someone else's job or has no logic |

## Which test catches which bug

| THE BUG YOU FEAR | CHEAPEST TEST | PAGE |
|---|---|---|
| A rule computes the wrong number | **unit test** | p.06 |
| SQL or mapping is wrong | **integration test** | p.19 |
| Schema change breaks old data | **migration test** | p.21 |
| Another service is slow or down | **WireMock stub** | p.30 |
| An API change breaks a consumer | **contract test** | p.31 |
| A duplicate message is applied twice | **integration test** | p.26 |
| The wrong user can see an order | **security + auth tests** | p.29 |
| Layers get tangled over time | **architecture test** | p.45 |
| The checkout button stops working | **end-to-end test** | p.34 |
| It is too slow under load | **performance test** | p.35 |

## Assert the outcome, not the steps

| BEHAVIOUR | FRAGILE: TIED TO HOW IT WORKS | ROBUST: CHECKS WHAT IT DOES |
|---|---|---|
| **Order total** | verify `discountPolicy.percentFor()` was called once | assert the total is ₹1,140.00 |
| **Order saved** | verify the exact SQL string sent to the database | read the order back and check its fields |
| **E-mail sent** | verify a private `formatBody()` helper ran | assert the received mail's recipient and subject |
| **Payment before saving** | — (here the *order* of calls is the rule) | `InOrder`: charge, then save (p.15) |

Interaction checks are right when the interaction *is* the behaviour — a payment charged, an e-mail requested, a message published — and wrong when they pin down internal steps.

**RULE OF THUMB**

Before writing a test, ask: **what bug would this catch**, and is there a cheaper test that catches it already? If you can't name the bug, the test is probably testing the framework, the language — or nothing.

01 · FOUNDATIONS

# Designing test cases

You can't test every input, so pick the few that are most likely to find bugs. The rule: `DiscountPolicy` gives 0% below ₹1000, 5% from ₹1000 and 10% from ₹5000.

● = a value tested in DiscountPolicyTest — one inside each partition plus both sides of every boundary




DiscountPolicyTest.java

```
@ParameterizedTest(name = "₹{0} → {1}% off",
                   quoteTextArguments = false)
@CsvSource({
  "0,       0",
  "999.99,  0",   // just below the first boundary
  "1000,    5",   // on the boundary
  "1000.01, 5",
  "4999.99, 5",
  "5000,   10",   // on the second boundary
  "250000, 10"
})
```

test output · JUnit 6.1.3 — 7 / 7 passed

```
└ percentForSubtotal(String, int) ✔
   ├ ₹0 → 0% off ✔
   ├ ₹999.99 → 0% off ✔
   ├ ₹1000 → 5% off ✔
   ├ ₹1000.01 → 5% off ✔
   ├ ₹4999.99 → 5% off ✔
   ├ ₹5000 → 10% off ✔
   └ ₹250000 → 10% off ✔
```

Mistype `≥` as `>` and only the **₹1000** row fails — see the mutation demo on p.39. Middle-of-range values like ₹2000 would never notice.

## Decision table for `placeOrder`

| RULE | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| **Items present?** | no | yes | yes | yes |
| **Payment approved?** | — | no | yes | yes |
| **Subtotal ≥ ₹1000?** | — | — | no | yes |
| **Outcome** | reject, call nobody | throw, save nothing | save, email, 0% | save, email, 5–10% |
| **Test** | `emptyOrderIsRejected…` | `declinedPayment…` | `chargesBeforeSaving` | `chargesDiscounted…` |

## Test design techniques

| TECHNIQUE | HOW | BEST FOR |
|---|---|---|
| **Equivalence partitioning** | Split inputs into groups the code treats the same; test one value from each. | Fewer tests, same coverage of behaviour |
| **Boundary value analysis** | Test on and either side of every edge: 999.99, 1000, 1000.01. | Catches off-by-one and `>` vs `≥` |
| **Decision table** | List every combination of conditions and the expected action. | Business rules with several inputs |
| **State transition** | Test each allowed move between states — and the forbidden ones. | Orders, payments, workflows |
| **Pairwise** | Cover every *pair* of option values instead of every combination. | Config and browser/OS matrices |
| **Property-based** | State a rule ("total ≥ 0") and let a tool generate hundreds of inputs. | Parsers, maths, serialisation (jqwik) |
| **Error guessing** | Use experience: null, empty, huge, Unicode, duplicates, time zones. | Finds what specs forget |

## Inputs worth trying on almost anything

**Empty & missing**
`null`, `""`, empty list, missing JSON field, zero items.

**Extremes**
`0`, `-1`, `Integer.MAX_VALUE`, 10 MB strings, 10,000 rows.

**Text**
Unicode (₹, emoji, Hindi), leading spaces, mixed case, SQL and HTML characters.

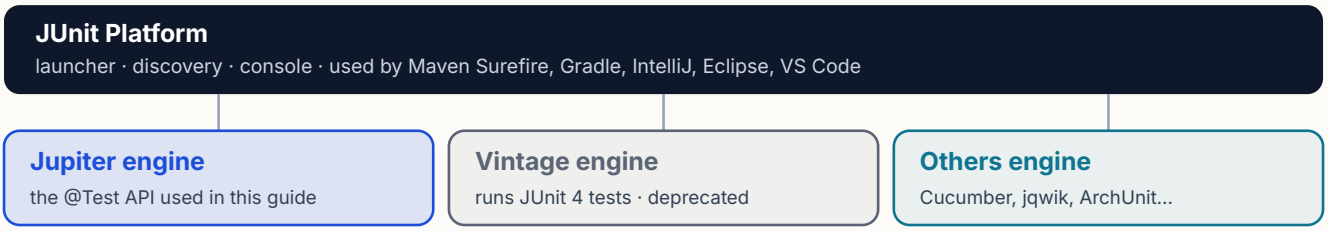
**Time**
Midnight, 29 Feb, month end, DST change, a different time zone.

CHAPTER TWO

# 02 Unit testing with JUnit 6

JUnit is the standard test framework for Java. Version 6 (30 September 2025) keeps the JUnit 5 programming model — the `org.junit.jupiter` API — and raises the floor to Java 17. Every test in this guide ran on JUnit 6.1.3, Mockito 5.24.0 and AssertJ 3.27.7 on Java 21.

## How the pieces fit




## Maven

pom.xml — test dependencies

```xml
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>6.1.3</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.assertj</groupId>
    <artifactId>assertj-core</artifactId>
    <version>3.27.7</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.24.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

## Gradle (Kotlin DSL)

build.gradle.kts

```kotlin
dependencies {
  testImplementation(platform("org.junit:junit-bom:6.1.3"))
  testImplementation("org.junit.jupiter:junit-jupiter")
  testRuntimeOnly("org.junit.platform:junit-platform-launcher")
  testImplementation("org.assertj:assertj-core:3.27.7")
  testImplementation("org.mockito:mockito-junit-jupiter:5.24.0")
}

tasks.test {
  useJUnitPlatform()
}
```

## Coming from JUnit 4

| JUNIT 4 | JUNIT 5 / 6 |
|---|---|
| `@Before / @After` | `@BeforeEach / @AfterEach` |
| `@BeforeClass / @AfterClass` | `@BeforeAll / @AfterAll` |
| `@Ignore` | `@Disabled` |
| `@Category(Slow.class)` | `@Tag("slow")` |
| `@RunWith(…)` | `@ExtendWith(…)` |
| `@Rule / @ClassRule` | `@ExtendWith or @RegisterExtension` |
| `@Test(expected = X.class)` | `assertThrows(X.class, …)` |
| `@Test(timeout = 500)` | `@Timeout(value = 500, unit = MILLISECONDS)` |
| `assertEquals(msg, exp, act)` | `assertEquals(exp, act, msg) — message last` |
| `public class / public void` | `package-private is enough` |

`junit-jupiter` bundles the API, the parameterized-test module and the engine. `mockito-junit-jupiter` brings `mockito-core` with it. Run with `mvn test` or `./gradlew test`; one class with `mvn test -Dtest=MoneyTest`.

02 · UNIT TESTING WITH JUNIT 6

# Your first test & the lifecycle

MoneyTest.java ✔ 3 / 3 passed

```
class MoneyTest {

  @Test
  @DisplayName("10% off ₹999.99 rounds half-up to ₹899.99")
  void percentOffRoundsHalfUp() {
    Money price = Money.of("999.99");  // Arrange

    Money discounted = price.percentOff(10);  // Act

    assertEquals(Money.of("899.99"), discounted); // Assert
  }

  @Test
  void rejectsNegativeAmounts() {
    var error = assertThrows(IllegalArgumentException.class,
        () -> Money.of("-1"));
    assertEquals("negative amount: -1", error.getMessage());
  }

  // addsAndMultiplies() is shown on p.09
}
```

The class under test is a small `Money` record (₹, two decimals). Test classes and methods can be package-private; JUnit finds any method annotated with `@Test`.

## The annotations you'll use daily

| | |
|---|---|
| `@Test` | A test method. No return value, not private. |
| `@ParameterizedTest` | Runs once per set of arguments (see p.10). |
| `@BeforeEach` / `@AfterEach` | Run around **every** test: fresh fixtures, clean-up. |
| `@BeforeAll` / `@AfterAll` | Once per class; `static` unless `@TestInstance(PER_CLASS)`. |
| `@DisplayName` | Readable name in reports, any text or emoji. |
| `@Nested` | Inner class that groups related tests and shares setup. |
| `@Tag("slow")` | Label for filtering what runs, e.g. in CI stages. |
| `@Disabled("why")` | Skip, with a reason. Prefer fixing or deleting. |
| `@TempDir` | Inject a fresh temporary directory, deleted afterwards. |
| `@ExtendWith` | Plug in an extension, e.g. `MockitoExtension`. |

## Lifecycle, observed

LifecycleTest.java

```
class LifecycleTest {
  static void log(String s) { System.out.println(s); }

  LifecycleTest() { log("  new LifecycleTest()"); }

  @BeforeAll static void beforeAll() { log("@BeforeAll"); }
  @BeforeEach void beforeEach()      { log("  @BeforeEach"); }

  @Test void first()                 { log("    first()"); }
  @Test void second()                { log("    second()"); }

  @AfterEach void afterEach()        { log("  @AfterEach"); }
  @AfterAll static void afterAll()   { log("@AfterAll"); }
}
```

real output

```
@BeforeAll
  new LifecycleTest()
  @BeforeEach
    second()
  @AfterEach
  new LifecycleTest()
  @BeforeEach
    first()
  @AfterEach
@AfterAll
```

- **A new instance per test method**, so fields can't leak state between tests.
- **Order is deterministic but deliberately not obvious** — here `second()` ran first. Never depend on it.
- Need an order? `@TestMethodOrder(OrderAnnotation.class)` — mostly for slow, staged integration tests.
- `@TestInstance(PER_CLASS)` reuses one instance, so `@BeforeAll` need not be static.

## What changed in JUnit 6

| CHANGE | WHAT IT MEANS FOR YOU |
|---|---|
| **Java 17 minimum** | Projects on older JDKs stay on JUnit 5.14.x until they upgrade. |
| **One version for everything** | Platform, Jupiter and Vintage now share one number, so the BOM is simpler. |
| **Vintage deprecated** | JUnit 4 tests still run, but the engine flags them at discovery; plan the migration. |
| **Quoted text arguments** | Parameterized names show `"1000"`; turn off with `quoteTextArguments = false`. |
| **CSV parsing via FastCSV** | `@CsvSource` is stricter: characters after a closing quote are now an error. |

02 · UNIT TESTING WITH JUNIT 6

# Assertions & AssertJ

An assertion is the test's oracle. JUnit's `Assertions` class covers the basics; **AssertJ** adds a fluent style — type `assertThat(x).` and your IDE lists every check that makes sense for `x` — and much clearer failure messages.

## JUnit assertions

| | |
|---|---|
| `assertEquals(expected, actual)` | Uses `equals()`. Expected first! |
| `assertTrue / assertFalse` | For booleans; message explains little. |
| `assertNull / assertNotNull` | Null checks. |
| `assertSame` | Same object reference (`==`). |
| `assertIterableEquals` | Same elements, same order. |
| `assertThrows(Type, code)` | Returns the exception for further checks. |
| `assertThrowsExactly` | Exact type, not a subclass. |
| `assertDoesNotThrow` | Documents that code must not fail. |
| `assertAll(…)` | Runs every check and reports all failures. |
| `assertTimeout(Preemptively)` | Fails when slower than a limit. |
| `assertInstanceOf` | Type check that returns the cast value. |

## Exceptions and grouped checks

MoneyTest.java — ✔ both pass

```
@Test
void rejectsNegativeAmounts() {
  var error = assertThrows(IllegalArgumentException.class,
      () → Money.of("-1"));
  assertEquals("negative amount: -1", error.getMessage());
}

@Test
void addsAndMultiplies() {
  Money m = Money.of("12.50");
  assertAll("arithmetic",
      () → assertEquals(Money.of("25.00"), m.plus(m)),
      () → assertEquals(Money.of("37.50"), m.times(3)),
      () → assertEquals("₹12.50", m.toString()));
}
```

## The same ideas in AssertJ

ItemTest.java

```
void rejected(int qty) {
  assertThatThrownBy(() → new Item("SKU-1", Money.of("10"), qty))
    .isInstanceOf(IllegalArgumentException.class)
    .hasMessageContaining("quantity");
}
```

## Failure messages are the real product

A test earns its keep the day it fails. These are the exact messages JUnit and AssertJ printed when we broke an expectation on purpose:

assertEquals

```
assertEquals(Money.of("900.00"),
    Money.of("999.99").percentOff(10));
```

JUnit says

```
expected: <₹900.00> but was: <₹899.99>
```

assertAll

```
assertAll("arithmetic",
  () → assertEquals(Money.of("25.00"), m.times(3)),
  () → assertEquals("Rs 12.50", m.toString()));
```

JUnit says — both failures

```
arithmetic (2 failures)
  AssertionFailedError: expected: <₹25.00> but was: <₹37.50>
  AssertionFailedError: expected: <Rs 12.50> but was: <₹12.50>
```

AssertJ

```
assertThat(List.of("PEN", "BOOK", "TV"))
    .containsExactly("PEN", "TV", "BOOK");
```

AssertJ says

```
Expecting actual:
  ["PEN", "BOOK", "TV"]
to contain exactly (and in same order):
  ["PEN", "TV", "BOOK"]
but there were differences at these indexes:
  - element at index 1: expected "TV" but was "BOOK"
  - element at index 2: expected "BOOK" but was "TV"
```

**WATCH OUT**

Put **expected first** in `assertEquals(expected, actual)`. Swapped arguments still fail correctly, but the message lies about which value was wrong. AssertJ avoids the problem: `assertThat(actual).isEqualTo(expected)`.

**TIP**

Compare **money with** `BigDecimal` and a fixed scale. `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is `false`; that's why `Money` always calls `setScale(2)`.

02 · UNIT TESTING WITH JUNIT 6

# Parameterized, nested & more

When several tests differ only in their data, write one parameterized test. Each row becomes its own entry in the report, so you see exactly which case failed.

ItemTest.java — @Nested + @MethodSource + @ValueSource

```java
@DisplayName("Item")
class ItemTest {

  @Nested @DisplayName("when quantity is valid")
  class ValidQuantity {
    @ParameterizedTest(name = "{0} × ₹{1} = ₹{2}",
            quoteTextArguments = false)
    @MethodSource("shop.ItemTest#totals")
    void totalIsPriceTimesQty(int qty, String price, String total) {
      assertThat(new Item("SKU-1", Money.of(price), qty).total())
        .isEqualTo(Money.of(total));
    }
  }

  @Nested @DisplayName("when quantity is invalid")
  class InvalidQuantity {
    @ParameterizedTest(name = "quantity {0} is rejected")
    @ValueSource(ints = {0, -1, Integer.MIN_VALUE})
    void rejected(int qty) {
      assertThatThrownBy(() -> new Item("SKU-1", Money.of("10"), qty))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("quantity");
    }
  }

  static Stream<Arguments> totals() {
    return Stream.of(
      Arguments.of(1, "10", "10"),
      Arguments.of(3, "19.99", "59.97"),
      Arguments.of(1000, "0.01", "10"));
  }
}
```

real output 6 / 6 passed

```
Item ✔
├─ when quantity is invalid ✔
│  └─ rejected(int) ✔
│     ├─ quantity 0 is rejected ✔
│     ├─ quantity -1 is rejected ✔
│     └─ quantity -2147483648 is rejected ✔
└─ when quantity is valid ✔
   └─ totalIsPriceTimesQty(int, String, String) ✔
      ├─ 1 × ₹10 = ₹10 ✔
      ├─ 3 × ₹19.99 = ₹59.97 ✔
      └─ 1000 × ₹0.01 = ₹10 ✔
```

**JUNIT 6 CHANGE**

JUnit 6 prints text arguments in quotes, and `@CsvSource` values count as text — even numbers. `quoteTextArguments = false` keeps names like **₹1000 → 5% off**.

**RULE OF THUMB**

Keep each row **one behaviour**. If rows need different assertions, they are different tests.

## Argument sources

| | |
|---|---|
| `@ValueSource(ints = {0, -1})` | One literal argument per run |
| `@NullAndEmptySource` | Adds `null` and `""` /empty collection |
| `@EnumSource(Status.class)` | Every enum constant, or a filtered set |
| `@CsvSource({"a, 1"})` | Several arguments per row, inline |
| `@CsvFileSource("/cases.csv")` | Rows from a CSV file on the classpath |
| `@MethodSource("cases")` | A static factory returning `Stream<Arguments>` |
| `@FieldSource("cases")` | A static field holding the arguments (JUnit 5.11+) |
| `@ArgumentsSource(My.class)` | Your own reusable `ArgumentsProvider` |

## More tools in the box

**Tags & filtering**

`@Tag("slow")` on a class or method, then `mvn test -Dgroups=fast` or `-DexcludedGroups=slow`.

**Assumptions**

`assumeTrue(isCi())` aborts (skips) the test instead of failing it when a precondition doesn't hold.

**Conditional tests**

`@EnabledOnOs(LINUX)`, `@EnabledForJreRange(min = JAVA_21)`, `@EnabledIfEnvironmentVariable`.

**Timeouts**

`@Timeout(5)` on a method or class; `assertTimeoutPreemptively` for one block (see p.32).

**Temp files**

`@TempDir Path dir` gives each test a fresh folder and deletes it afterwards.

**Extensions**

`@ExtendWith` / `@RegisterExtension` hook into the lifecycle: Mockito, Spring, Testcontainers all work this way.

02 · UNIT TESTING WITH JUNIT 6

# Writing your own JUnit extensions

When the same setup appears in many test classes — a fixed clock, a timer, a test database — turn it into an **extension**. Extensions plug into the test lifecycle through small interfaces; a composed annotation then gives the whole team one line to write.

FixedClockExtension — a ParameterResolver

```
/** Resolves a Clock parameter from @FixedClock on the test. */
public class FixedClockExtension implements ParameterResolver {
  @Override
  public boolean supportsParameter(ParameterContext p,
      ExtensionContext ctx) {
    return p.getParameter().getType() == Clock.class;
  }

  @Override
  public Object resolveParameter(ParameterContext p,
      ExtensionContext ctx) {
    FixedClock fixed = AnnotationSupport.findAnnotation(
            ctx.getRequiredTestMethod(), FixedClock.class)
        .or(() -> AnnotationSupport.findAnnotation(
            ctx.getRequiredTestClass(), FixedClock.class))
        .orElseThrow(() -> new ParameterResolutionException(
            "Clock parameter needs @FixedClock"));
    return Clock.fixed(Instant.parse(fixed.value()),
        ZoneOffset.UTC);
  }
}
```

TimingExtension — lifecycle callbacks + the Store

```
/** Reports how long each test method took. */
public class TimingExtension implements
    BeforeTestExecutionCallback, AfterTestExecutionCallback {
  private static final Namespace NS =
      Namespace.create(TimingExtension.class);

  @Override
  public void beforeTestExecution(ExtensionContext ctx) {
    ctx.getStore(NS).put("start", System.nanoTime());
  }

  @Override
  public void afterTestExecution(ExtensionContext ctx) {
    long start = ctx.getStore(NS).remove("start", Long.class);
    long ms = (System.nanoTime() - start) / 1_000_000;
    System.out.printf("[timing] %s took %d ms%n",
        ctx.getDisplayName(), ms);
  }
}
```

ShopTest — a composed annotation

```
/** One annotation for the team: tag + both extensions. */
```

DeliveryDateTest — using it

```
@ShopTest
@FixedClock("2026-10-16T09:00:00Z")          // a Friday
class DeliveryDateTest {

  /** Ships in 2 working days; weekends don't count. */
  static LocalDate deliveryDate(Clock clock) {
    LocalDate day = LocalDate.now(clock);
    for (int left = 2; left > 0; ) {
      day = day.plusDays(1);
      if (day.getDayOfWeek().getValue() < 6) left--;
    }
    return day;
  }

  @Test
  void fridayOrderArrivesTuesday(Clock clock) {
    assertThat(deliveryDate(clock)).isEqualTo("2026-10-20");
  }

  // … @RepeatedTest(3), TestInfo — see the repo
}
```

real output · Surefire 3.6.0

```
[timing] fridayOrderArrivesTuesday(Clock) took 4 ms
[timing] repetition 1 of 3 took 2 ms
[timing] repetition 2 of 3 took 0 ms
[timing] repetition 3 of 3 took 0 ms
[timing] tuesdayOrderArrivesThursday(Clock, TestInfo) took 1 ms
Tests run: 5, Failures: 0
```

## Extension points you will use

| INTERFACE | USE IT TO |
|---|---|
| `BeforeAllCallback`, `BeforeEachCallback`, … `TestExecutionCallback`, `After…` | set up and tear down around tests — like `@BeforeEach`, but reusable |
| `ParameterResolver` | inject arguments into test methods and constructors |
| `ExecutionCondition` | decide whether a test runs at all |
| `TestExecutionExceptionHandler` | react to — or swallow — exceptions thrown by tests |
| `TestWatcher` | hear about results: passed, failed, aborted, disabled |
| `InvocationInterceptor` | wrap the call itself, e.g. run it on another thread |

- **Register** with `@ExtendWith` (as above), a `static @RegisterExtension` field when you need to configure it, or automatically through `ServiceLoader` with `junit.jupiter.extensions.autodetection.enabled=true`.
- **Keep state in the `ExtensionContext.Store`**, never in fields: one extension instance serves many tests, possibly in parallel.
- `TestInfo` and `RepetitionInfo` are built-in parameter resolvers — that is how `@RepeatedTest` tells you which run it is.
- Look for an existing extension first: Mockito, Spring, Testcontainers and many libraries ship one.

02 · UNIT TESTING WITH JUNIT 6

# Testing logs and metrics

On-call engineers read your log lines, and alerts fire on your metrics. If a refactoring silently drops a warning or renames a counter, production monitoring goes blind — so assert on them like any other output.

PaymentService.java

```java
public class PaymentService {
  private static final Logger log =
      LoggerFactory.getLogger(PaymentService.class);
  static final long LIMIT_PAISE = 5_000_000;     // ₹50,000

  private final Counter declined;

  public PaymentService(MeterRegistry registry) {
    declined = Counter.builder("payments.declined")
        .tag("reason", "over_limit")
        .register(registry);
  }

  public boolean charge(String orderId, long amountPaise) {
    if (amountPaise > LIMIT_PAISE) {
      declined.increment();
      log.warn("Payment declined for order {}: over limit",
          orderId);
      return false;
    }
    log.info("Charged order {}", orderId);
    return true;
  }
}
```

PaymentServiceTest — Logback ListAppender + Micrometer — 2 ✔ · 0.056 s

```java
class PaymentServiceTest {
  SimpleMeterRegistry registry = new SimpleMeterRegistry();
  ListAppender<ILoggingEvent> logs = new ListAppender<>();
  Logger logger =
      (Logger) LoggerFactory.getLogger(PaymentService.class);

  @BeforeEach void capture() {
    logs.start();
    logger.addAppender(logs);
  }
  @AfterEach void release() { logger.detachAppender(logs); }

  @Test
  void declinedPaymentIsLoggedAndCounted() {
    var payments = new PaymentService(registry);

    assertThat(payments.charge("ord-9", 6_000_000)).isFalse();

    assertThat(logs.list)
        .extracting(ILoggingEvent::getLevel,
                    ILoggingEvent::getFormattedMessage)
        .containsExactly(tuple(Level.WARN,
            "Payment declined for order ord-9: over limit"));
    assertThat(registry.get("payments.declined")
        .tag("reason", "over_limit").counter().count())
        .isEqualTo(1.0);
  }

  @Test
  void normalPaymentRaisesNoAlarm() {
    var payments = new PaymentService(registry);

    assertThat(payments.charge("ord-1", 100_000)).isTrue();

    assertThat(logs.list)
        .noneMatch(e -> e.getLevel() == Level.WARN);
    assertThat(registry.get("payments.declined")
        .counter().count()).isZero();
  }
}
```

### Logs: capture, don't parse files

Attach Logback's `ListAppender` to the logger under test and assert on level and message. Detach it afterwards so tests stay independent.

### Metrics: a real registry

Micrometer's `SimpleMeterRegistry` is an in-memory registry made for tests. Assert on the meter name and tags your dashboards and alerts use.

## Tools for each signal

| SIGNAL | TEST TOOL | ASSERT ON |
|---|---|---|
| **Logs (Logback)** | `ListAppender` attached to the logger | level, message or key, MDC values |
| **Logs (Spring Boot)** | `OutputCaptureExtension` → `CapturedOutput` | text written to stdout / stderr |
| **Metrics (Micrometer)** | `SimpleMeterRegistry` | meter name, tags, count or total |
| **Traces (OpenTelemetry)** | `InMemorySpanExporter` from opentelemetry-sdk-testing | span names, attributes, parent/child links |

RULE OF THUMB

Assert on what operators depend on — level, a stable message or key, metric name and tags — not on whole formatted lines with timestamps. Otherwise every log-format change breaks tests.

CHAPTER THREE

# 03 Test doubles & Mockito

A unit test shouldn't send real e-mails or charge real cards. A **test double** stands in for a collaborator, like a stunt double for an actor. The five kinds come from Gerard Meszaros's *xUnit Test Patterns* (2007).

## The five test doubles

| DOUBLE | WHAT IT IS | WHAT YOU VERIFY | IN THIS GUIDE |
|---|---|---|---|
| `dummy` | Fills a parameter list; never actually used. | Nothing | `dummyEmail` throws if touched |
| `stub` | Returns canned answers to calls. | State of the SUT | `alwaysApproves` payment gateway |
| `spy` | A stub that also records the calls it got. | Behaviour, checked by you | `SpyEmailSender.sentTo` |
| `mock` | Pre-programmed with expectations; the library checks them. | Behaviour | Mockito `verify(email).send(…)` |
| `fake` | A working, simplified implementation. | State, realistically | `InMemoryOrderRepository` |

TestDoublesTest.java — no library needed ✔ 3 / 3 passed

```java
/** The five kinds of test double, hand-written — no library needed. */
class TestDoublesTest {

  // DUMMY: passed in to satisfy a parameter, never used
  EmailSender dummyEmail = (to, subject, body) -> {
    throw new AssertionError("not used");
  };

  // STUB: returns canned answers
  PaymentGateway alwaysApproves = (customer, amount) -> "PAY-1";

  // SPY: a stub that also records how it was called
  static class SpyEmailSender implements EmailSender {
    final List<String> sentTo = new ArrayList<>();
    @Override public void send(String to, String subject, String body) {
      sentTo.add(to);
    }
  }

  // FAKE: a real but simplified implementation (a map instead of a DB)
  OrderRepository fakeRepo = new InMemoryOrderRepository();

  List<Item> pens = List.of(new Item("PEN", Money.of("50"), 2));

  OrderService serviceWith(EmailSender email) {
    return new OrderService(fakeRepo, alwaysApproves, email,
        new DiscountPolicy(), Clock.systemUTC());
  }

  @Test
  void stubAndFakeLetUsCheckState() {
    var service = serviceWith(new SpyEmailSender());
    Order order = service.placeOrder("meera@example.com", pens);
    assertThat(fakeRepo.findById(order.id())).contains(order);  // state
  }

  @Test
  void spyLetsUsCheckBehaviour() {
    var spy = new SpyEmailSender();
    serviceWith(spy).placeOrder("meera@example.com", pens);
    assertThat(spy.sentTo)
        .containsExactly("meera@example.com");  // behaviour
  }

  @Test
  void dummyIsNeverTouchedWhenValidationFails() {
    var service = serviceWith(dummyEmail);
    assertThatIllegalArgumentException()
      .isThrownBy(() -> service.placeOrder("x@example.com", List.of()));
  }
}
```

## State or behaviour?

### State verification

Run the code, then look at the result: what was returned, or what is now stored in the fake. Survives refactoring well, because it doesn't care *how* the result was reached.

### Behaviour verification

Check which calls were made to collaborators: "an e-mail was sent to Meera". Needed when the call *is* the result — sending, publishing, charging — and nothing observable is stored.

03 · TEST DOUBLES & MOCKITO

# Mockito in practice

`placeOrder` applies the discount, charges the card, saves the order and e-mails the customer. In the test the gateway, repository and mailer are mocks; the clock is fixed.

OrderService.java — under test

```
public Order placeOrder(String customerEmail, List<Item> items) {
  if (items.isEmpty())
    throw new IllegalArgumentException("order has no items");
  Money subtotal = items.stream().map(Item::total)
      .reduce(Money.of("0"), Money::plus);
  Money total = subtotal.percentOff(discounts.percentFor(subtotal));
  payments.charge(customerEmail, total);  // may throw PaymentDeclined…
  Order order = repository.save(new Order(UUID.randomUUID().toString(),
      customerEmail, List.copyOf(items), total, clock.instant()));
  email.send(customerEmail, "Order confirmed", "Total: " + total);
  return order;
}
```

- Every collaborator arrives through the **constructor**, so a test can pass in doubles — no framework needed.
- Time comes from an injected `java.time.Clock`, so the timestamp is predictable.
- `placeOrder` has one happy path and two failure paths: no items, or a declined card.

OrderServiceTest.java (first test)

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

  @Mock OrderRepository repository;
  @Mock PaymentGateway payments;
  @Mock EmailSender email;
  @Captor ArgumentCaptor<Order> savedOrder;

  Clock clock = Clock.fixed(Instant.parse("2026-10-13T10:00:00Z"),
                            ZoneOffset.UTC);
  OrderService service;

  @BeforeEach
  void setUp() {
    service = new OrderService(repository, payments, email,
        new DiscountPolicy(), clock);
  }

  @Test
  void chargesDiscountedTotalSavesAndEmails() {
    when(repository.save(any())).thenAnswer(c -> c.getArgument(0));

    Order order = service.placeOrder("asha@example.com",
        List.of(new Item("BOOK", Money.of("600"), 2))); // ₹1200 → 5%

    assertThat(order.total()).isEqualTo(Money.of("1140"));
    verify(payments).charge("asha@example.com", Money.of("1140"));
    verify(repository).save(savedOrder.capture());
    assertThat(savedOrder.getValue().placedAt())
        .isEqualTo(clock.instant());
    verify(email).send(eq("asha@example.com"), eq("Order confirmed"),
        contains("₹1140.00"));
  }
```

real output — 4 / 4 passed

```
OrderServiceTest ✔
├─ chargesBeforeSaving() ✔
├─ chargesDiscountedTotalSavesAndEmails() ✔
├─ declinedPaymentSavesNothingAndSendsNoEmail() ✔
└─ emptyOrderIsRejectedEarly() ✔
```

- **@ExtendWith(MockitoExtension.class)** creates the `@Mock` and `@Captor` fields before each test and checks for unused stubs after it.
- **thenAnswer** makes `save` return its argument, like a real repository.
- **ArgumentCaptor** grabs the saved `Order` so we can check its timestamp came from the fixed clock.
- **eq** and **contains** are matchers: one matcher means all arguments need one.

### Constructor beats @InjectMocks

`@InjectMocks` builds the object by constructor, setter or field injection — and fails *silently* if it can't. Calling the constructor yourself is explicit and breaks loudly.

### Defaults of a mock

An unstubbed mock returns empty values: `null`, `0`, `false`, empty collections and `Optional.empty()`. Void methods simply do nothing.

### Stub less, verify less

Stub only what the code path needs and verify only calls that are the point of the test — the rest is noise that breaks on refactors.

**Classicist** *vs* **Mockist**

Use real objects where possible, doubles only for awkward ones (network, time, mail).

Use a mock for any collaborator with interesting behaviour, and verify the calls made to it.

From Martin Fowler's *Mocks Aren't Stubs* (2004, revised 2007).

GOOD TO KNOW

Mockito's `mock()` can be a stub, spy or mock depending on use — the library blurs Meszaros's names. The five kinds still help you decide what to check.

03 · TEST DOUBLES & MOCKITO

# Failure paths, call order & the API

OrderServiceTest.java — failure path and call order

```java
@Test
void declinedPaymentSavesNothingAndSendsNoEmail() {
  when(payments.charge(anyString(), any()))
    .thenThrow(new PaymentDeclinedException("card expired"));

  assertThatThrownBy(() -> service.placeOrder("asha@example.com",
      List.of(new Item("PEN", Money.of("50"), 1))))
    .isInstanceOf(PaymentDeclinedException.class)
    .hasMessage("card expired");

  verifyNoInteractions(repository, email);
}

@Test
void chargesBeforeSaving() {
  when(repository.save(any())).thenAnswer(c -> c.getArgument(0));

  service.placeOrder("asha@example.com",
      List.of(new Item("PEN", Money.of("50"), 1)));

  InOrder inOrder = inOrder(payments, repository, email);
  inOrder.verify(payments).charge(any(), any());
  inOrder.verify(repository).save(any());
  inOrder.verify(email).send(any(), any(), any());
}
```

## The API on one table

| | |
|---|---|
| **Create** | `mock(X.class)` · `@Mock` · `spy(obj)` · `@Spy` · `@Captor` · `@InjectMocks` |
| **Stub** | `when(m.call()).thenReturn(v)` · `.thenThrow(e)` · `.thenAnswer(inv → …)` |
| **Stub void / spies** | `doThrow(e).when(m).call()` · `doReturn(v).when(spy).call()` · `doNothing()` |
| **Verify** | `verify(m).call()` · `times(2)` · `never()` · `atLeastOnce()` · `timeout(100)` · `verifyNoInteractions(m)` |
| **Order** | `InOrder o = inOrder(a, b);` `o.verify(a).x(); o.verify(b).y();` |
| **Match** | `any()` · `anyString()` · `eq(v)` · `contains(s)` · `argThat(x → …)` · `isNull()` |
| **Capture** | `verify(m).save(captor.capture());` `captor.getValue()` |
| **BDD style** | `given(m.call()).willReturn(v)` · `then(m).should().call()` |

## Given / when / then with BDDMockito

OrderServiceBddTest.java — ✔ passes

```java
  @Test
  void bigOrderGetsTenPercentOff() {
    // given
    given(repository.save(any())).willAnswer(c -> c.getArgument(0));
    var service = new OrderService(repository, payments, email,
        new DiscountPolicy(), Clock.systemUTC());

    // when
    Order order = service.placeOrder("ravi@example.com",
        List.of(new Item("TV", Money.of("5000"), 1)));

    // then
    assertThat(order.total()).isEqualTo(Money.of("4500"));
    then(payments).should().charge("ravi@example.com", Money.of("4500"));
    then(email).should(times(1)).send(any(), any(), any());
  }
}
```

`BDDMockito` renames the same calls so tests read like a specification: `given(…).willReturn(…)` instead of `when(…).thenReturn(…)`, and `then(mock).should()` instead of `verify(mock)`. Pick one style per code base: mixing them makes readers wonder whether they behave differently — they don't.

## Rules that keep mocks honest

**Don't mock what you don't own**

Wrap a third-party client (Stripe, S3, an HTTP SDK) in your own small interface and mock *that*. Test the wrapper against the real thing in an integration test.

**Don't mock values**

Records, `Money`, `List`, `LocalDate`: just create real ones. Mocking them makes tests longer and hides bugs.

**Matchers: all or nothing**

If one argument uses a matcher, every argument must: `eq("a"), any()`, not `"a", any()` — Mockito throws `InvalidUseOfMatchersException`.

**A mock returning a mock is a smell**

`when(a.getB().getC())` means the code reaches through objects. Pass what it needs directly.

**Verify outcomes, not every call**

`verifyNoMoreInteractions` on every test makes harmless refactors fail. Verify the calls that matter.

**Too many mocks**

Five or more mocks in one test means the class has many collaborators. Split it, or test a bigger unit with fakes.

03 · TEST DOUBLES & MOCKITO

# Spies, statics, errors & pitfalls

MockitoFeaturesTest.java ✔ 4 / 4 passed

```java
class MockitoFeaturesTest {

  @Test
  void consecutiveAnswersForRetries() {
    PaymentGateway gateway = mock(PaymentGateway.class);
    when(gateway.charge(any(), any()))
      .thenThrow(new PaymentDeclinedException("timeout"))
      .thenReturn("PAY-2");

    Money one = Money.of("1");
    assertThatThrownBy(() -> gateway.charge("a@example.com", one));
    assertThat(gateway.charge("a@example.com", one)).isEqualTo("PAY-2");
    verify(gateway, times(2)).charge(any(), any());
  }

  @Test
  void spyWrapsARealObject() {
    List<String> list = spy(new ArrayList<>());
    list.add("one");
    // on spies use doReturn: when(list.size()) calls the real method
    doReturn(100).when(list).size();

    assertThat(list.size()).isEqualTo(100);
    assertThat(list.get(0)).isEqualTo("one");  // real behaviour kept
    verify(list).add("one");
  }

  @Test
  void mockStaticIsScopedToTheTryBlock() {
    UUID fixed = UUID.fromString("0-0-0-0-42");
    try (MockedStatic<UUID> uuid =
             mockStatic(UUID.class, CALLS_REAL_METHODS)) {
      uuid.when(UUID::randomUUID).thenReturn(fixed);
      assertThat(UUID.randomUUID()).isEqualTo(fixed);
    }
    assertThat(UUID.randomUUID()).isNotEqualTo(fixed); // real again
  }

  @Test
  void finalClassesCanBeMockedByDefault() {
    DiscountPolicy policy = mock(DiscountPolicy.class); // final class
    when(policy.percentFor(any())).thenReturn(50);
    assertThat(policy.percentFor(Money.of("10"))).isEqualTo(50);
  }
}
```

## Reading Mockito's errors

verify(payments).charge(any(), any());

```
Wanted but not invoked:
payments.charge(<any>, <any>);
→ at
shop.FailingExamplesTest.f_wantedButNotInvoked(Fail
ingExamplesTest.java:45)
Actually, there were zero interactions with this
mock.
```

verify(email).send(…, "Total: ₹1200.00");

```
Argument(s) are different! Wanted:
email.send(
    "asha@example.com",
    "Order confirmed",
    "Total: ₹1200.00"
);
→ at
shop.FailingExamplesTest.e_argumentMismatch(Failing
ExamplesTest.java:41)
Actual invocations have different arguments at
position [2]:
email.send(
    "asha@example.com",
    "Order confirmed",
    "Total: ₹1140.00"
);
→ at
shop.FailingExamplesTest.e_argumentMismatch(Failing
ExamplesTest.java:40)
```

a when(…) the test never used

```
Unnecessary stubbings detected.
Clean & maintainable test code requires zero
unnecessary code.
Following stubbings are unnecessary (click to
navigate to relevant line of code):
1. → at
shop.FailingExamplesTest.d_unnecessaryStubbing(Fail
ingExamplesTest.java:35)
Please remove unnecessary stubbings or use
'lenient' strictness. More info: javadoc for
UnnecessaryStubbingException class.
```

**Strictness:** `MockitoExtension` defaults to `STRICT_STUBS` — unused stubs fail the test. `WARN` only logs; `LENIENT` ignores them. Change it with `@MockitoSettings(strictness = …)` or per stub with `lenient().when(…)`. An unused `when(…)` is dead setup that misleads readers, so keep the default. The three messages above are real output from deliberately failing tests.

**JDK 21+ WARNING**

Mockito 5's default **inline mock maker** (final classes, `mockStatic`) attaches a Java agent at run time. JDK 21 warns about that; JEP 451 aims to disallow dynamic agent loading by default in a later release. Load Mockito as an agent in the build instead. `@{argLine}` must already be defined (JaCoCo's `prepare-agent` sets it; otherwise declare an empty `argLine` property).

pom.xml — Mockito as a -javaagent (from Mockito's docs)

```xml
<plugin>  <!-- defines ${org.mockito:mockito-core:jar} -->
  <artifactId>maven-dependency-plugin</artifactId>
  <executions>
    <execution><goals><goal>properties</goal></goals></execution>
  </executions>
</plugin>
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <argLine>@{argLine} -javaagent:${org.mockito:mockito-core:jar}</argLine>
  </configuration>
</plugin>
```

03 · TEST DOUBLES & MOCKITO

# Testing legacy code: characterization tests & seams

Code without tests is hard to change safely, and often hard to test without changing it first. Michael Feathers' *Working Effectively with Legacy Code* (2004) is a book about that problem; its chapters include "The Seam Model" and "Dependency-Breaking Techniques". The way out: make the **smallest** safe change that lets a test in, record what the code does, then improve it.

**1 · Find the change point**
Where must the new behaviour go?

**2 · Find a test point**
Where can a test observe what that code does — a return value, a file, a call?

**3 · Break dependencies**
Open a **seam** — a place to change behaviour without editing the code there: a constructor parameter, an overridable method.

**4 · Pin current behaviour**
Write **characterization tests**: they assert what the code does today, right or wrong.

**5 · Change and refactor**
Now change the code; the tests tell you what else moved.

InvoiceReport.java — the only change: a Clock seam

```java
/**
 * Legacy code, as found — except for ONE change: the Clock.
 * It used to call LocalDate.now() directly, so its output
 * changed every day and no test could pin it down.
 */
public class InvoiceReport {
  private final Clock clock;

  public InvoiceReport() { this(Clock.systemDefaultZone()); }

  public InvoiceReport(Clock clock) {   // the seam
    this.clock = clock;
  }

  /** rows: sku, quantity, unit price */
  public String render(List<String[]> rows) {
    StringBuilder out = new StringBuilder();
    out.append("INVOICE ").append(LocalDate.now(clock))
       .append('\n');
    double total = 0;
    for (String[] r : rows) {
      int qty = Integer.parseInt(r[1]);
      double price = Double.parseDouble(r[2]);
      double line = qty * price;
      if (qty >= 10) line = line * 0.95;     // bulk discount
      total += line;
      out.append(String.format("%-5s %3d x %7.2f = %8.2f%n",
          r[0], qty, price, line));
    }
    if (total > 1000) total = total - 50;    // ??? nobody knows
    out.append(String.format("TOTAL %.2f%n", total));
    return out.toString();
  }
}
```

characterization test — a "golden master" — 5 ✔ · 0.060 s

```java
@Test
void outputMatchesTheApprovedFile() throws Exception {
  String actual = new InvoiceReport(FIXED).render(List.of(
      new String[] {"BOOK", "2", "600.00"},
      new String[] {"PEN", "10", "12.50"},
      new String[] {"BAG", "1", "499.99"}));

  Path approved = Path.of("src/test/resources/approved",
                          "invoice.txt");
  if (!Files.exists(approved)) {          // first run: record
    Files.writeString(Path.of("target/received.txt"), actual);
    fail("Review target/received.txt, then approve it");
  }
```

src/test/resources/approved/invoice.txt

```
INVOICE 2026-10-13
BOOK     2 x  600.00 =  1200.00
PEN     10 x   12.50 =   118.75
BAG      1 x  499.99 =   499.99
TOTAL 1768.74
```

probing the edges — and pinning today's answers

```java
@ParameterizedTest(name = "{0} x {1} → {2}")
@CsvSource({       // edges found by probing; TODAY's values
    " 9, 100.00, 900.00",     // no bulk discount below 10
    "10, 100.00, 950.00",     // 5% off from 10
    " 1, 1000.00, 1000.00",   // exactly 1000: no 50 off
    " 1, 1000.01, 950.01",    // just above: 50 off
})
```

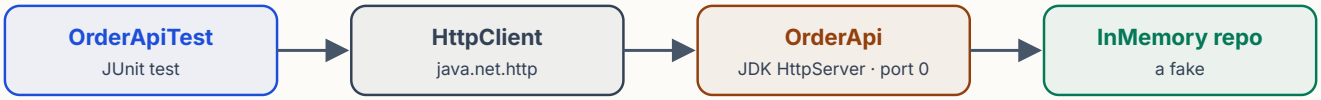
- **Record, don't judge.** "Totals over ₹1,000 lose ₹50" looks like a bug — but someone may rely on it. Pin it, then ask.
- **The seam:** `LocalDate.now()` made the output change daily. A constructor taking a `Clock` — old callers keep the no-argument one — made it testable without touching the logic.
- **Other seams:** subclass and override a method that does I/O; extract an interface for a static call; pass a function instead of calling a singleton.
- **Tools:** ApprovalTests automates the received/approved file workflow; Mockito `mockStatic` (p.16) is a last-resort seam for statics.

CHAPTER FOUR

# 04 Integration & API testing

Unit tests prove each class works alone; integration tests prove they work *together* — with real HTTP, serialisation, SQL and configuration. They are slower, so keep them focused.

## Testing a real HTTP API




OrderApiTest.java

```java
/** Integration test: a real HTTP server on a free port + a real client. */
class OrderApiTest {
  static final OrderRepository repo = new InMemoryOrderRepository();
  static final OrderApi api = new OrderApi(repo);
  static final HttpClient http = HttpClient.newHttpClient();
  static String baseUrl;

  @BeforeAll static void startServer() throws Exception {
    baseUrl = "http://127.0.0.1:" + api.start();
    repo.save(new Order("ord-1", "asha@example.com",
        List.of(new Item("BOOK", Money.of("600"), 2)),
        Money.of("1140"), Instant.EPOCH));
  }

  @AfterAll static void stopServer() { api.stop(); }

  HttpResponse<String> send(String method, String path)
      throws Exception {
    var request = HttpRequest.newBuilder(URI.create(baseUrl + path))
        .method(method, HttpRequest.BodyPublishers.noBody()).build();
    return http.send(request, HttpResponse.BodyHandlers.ofString());
  }

  @Test void existingOrderReturns200WithJson() throws Exception {
    var res = send("GET", "/orders/ord-1");
    assertThat(res.statusCode()).isEqualTo(200);
    assertThat(res.headers().firstValue("Content-Type"))
      .hasValue("application/json");
    assertThat(res.body())
      .isEqualTo("{\"id\":\"ord-1\",\"total\":\"1140.00\",\"items\":1}");
  }

  @Test void unknownOrderReturns404() throws Exception {
    assertThat(send("GET", "/orders/nope").statusCode()).isEqualTo(404);
  }

  @Test void wrongMethodReturns405WithAllowHeader() throws Exception {
    var res = send("DELETE", "/orders/ord-1");
    assertThat(res.statusCode()).isEqualTo(405);
    assertThat(res.headers().firstValue("Allow")).hasValue("GET");
  }

  // pathTricksAreJustNotFound() → p.36
}
```

real output — 4 / 4 passed

```
OrderApiTest ✔
├─ wrongMethodReturns405WithAllowHeader() ✔
├─ unknownOrderReturns404() ✔
├─ pathTricksAreJustNotFound() ✔
└─ existingOrderReturns200WithJson() ✔
```

- **Port 0** lets the OS pick a free port, so tests never clash with each other or with a running app.
- **@BeforeAll / @AfterAll** start the server once per class — starting it per test would be slow.
- Check the **status, headers and body**: a 200 with the wrong `Content-Type` is still a bug.
- Test the **unhappy paths** too: 404 for a missing order, 405 plus an `Allow` header for a wrong method.
- JDK `HttpServer` + `HttpClient` keep it dependency-free; Spring Boot's `RANDOM_PORT` gives the same shape.

## Narrow or broad?

**Narrow integration test** vs **Broad integration test**

Exercises *one* boundary — repository vs real database, client vs stubbed HTTP server.

Starts most of the application and checks a whole flow through several boundaries.

Prefer many narrow tests: their failures point straight at the broken boundary.

**In-memory stand-in** vs **The real thing in a container**

H2 for PostgreSQL, a map for Redis. Fast, but dialects and locking differ.

Testcontainers runs production's PostgreSQL version in Docker.

Fakes for *unit* tests; the real engine when the database is what you're testing.

04 · INTEGRATION & API TESTING

# Spring Boot, Testcontainers & contracts

In a Spring Boot application, a **test slice** starts only the part of the context one layer needs, so it is far faster than `@SpringBootTest`. Spring Boot 4.0 (November 2025) moved each slice into its own test module.

| ANNOTATION | LOADS | BOOT 4 MODULE |
|---|---|---|
| `@WebMvcTest` | Controllers, filters, JSON, validation; no services or DB | `spring-boot-webmvc-test` |
| `@WebFluxTest` | The same for reactive WebFlux controllers | `spring-boot-webflux-test` |
| `@DataJpaTest` | JPA repositories, entities, transactions (rolled back) | `spring-boot-data-jpa-test` |
| `@JdbcTest` | `JdbcTemplate` / `JdbcClient` code | `spring-boot-jdbc-test` |
| `@JsonTest` | Jackson / Gson (de)serialisation only | `spring-boot-test-autoconfigure` |
| `@RestClientTest` | Your HTTP clients against a mock server | `spring-boot-restclient-test` |
| `@SpringBootTest` | The whole application context — the broad test | `spring-boot-test` |

OrderControllerTest — controller slice

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
  @Autowired MockMvcTester mvc;       // AssertJ-style MockMvc
  @MockitoBean OrderService orders;   // replaces the real bean

  @Test
  void knownOrderIsReturnedAsJson() {
    given(orders.find("A-1")).willReturn(Optional.of(
        new OrderView("A-1", new BigDecimal("1140.00"), 2)));

    assertThat(mvc.get().uri("/orders/A-1"))
        .hasStatusOk()
        .bodyJson().isLenientlyEqualTo("""
            {"reference":"A-1","total":1140.00,"lines":2}""");
  }

  @Test
  void unknownOrderIs404() {
    given(orders.find("nope")).willReturn(Optional.empty());

    assertThat(mvc.get().uri("/orders/nope"))
        .hasStatus(HttpStatus.NOT_FOUND);
  }
}
```

TestcontainersConfiguration — PostgreSQL as a bean

```
@TestConfiguration(proxyBeanMethods = false)
class TestcontainersConfiguration {
  @Bean
  @ServiceConnection                  // Boot wires the JDBC URL
  PostgreSQLContainer postgres() {
    return new PostgreSQLContainer("postgres:17-alpine");
  }
}
```

real run · Spring Boot 4.1.1 on GitHub Actions

```
✔ OrderControllerTest  2 tests  1.943 s
✔ ArchitectureRulesCatchViolationsTest 1 tests  0.877 s
✔ ArchitectureTest     3 tests  0.354 s
✔ OrderRepositoryIT    2 tests  11.51 s
✔ OrderFlowIT          3 tests  2.471 s
```

Versions managed by Boot 4.1.1: Spring Framework 7.0.9, JUnit 6.0.3, Mockito 5.23.0, Hibernate 7.4.5.Final, Testcontainers 2.0.5. Project: `spring/` in the example code. Our first version used `static @Container` fields per class; that run's test JVM took 30 s to exit and Failsafe had to kill it. With the container as a Spring bean it closes with the context, and the JVM exited at once.

### @MockitoBean, not @MockBean

Spring Framework 6.2 added `@MockitoBean` and `@MockitoSpyBean`. Boot 3.4 deprecated `@MockBean` / `@SpyBean`, and Boot 4.0 removed them.

### @ServiceConnection

Since Boot 3.1: Boot sees the container's type and wires the connection details for you — no `spring.datasource.url` juggling.

### Testcontainers 2.0

October 2025. Module artifacts gained a prefix: `testcontainers-postgresql`, `testcontainers-junit-jupiter`.

## Calling other services: stubs and contracts

**WireMock** runs a real HTTP server that returns canned responses, so your client code sends real requests over a socket. Stub the slow, the broken and the odd: timeouts, 500s, malformed JSON, redirects.

**The risk:** your stub and the real provider drift apart, and both sides stay green while production breaks.

**Consumer-driven contract tests** (Ian Robinson, 2006) close that gap. The consumer records what it needs from the API; the provider's build replays those expectations against the real service. Tools: **Pact** (many languages, a broker to share contracts) and **Spring Cloud Contract** (JVM, contracts generate stubs and provider tests).

04 · INTEGRATION & API TESTING

# `@Transactional` in tests: what the rollback hides

`@DataJpaTest` — and any test class annotated `@Transactional` — runs each test inside a transaction that is **rolled back**. That keeps the database clean, but nothing is ever committed, so two kinds of production bug can't happen in the test. The same code, run both ways against real PostgreSQL:

rolled back · OrderRepositoryIT — 2 ✔

```
@DataJpaTest
@Import(TestcontainersConfiguration.class)
class OrderRepositoryIT {
  @Autowired OrderRepository repository;

  @Test
  void duplicateReferenceSlipsThroughUntilFlush() {
    repository.save(new CustomerOrder("A-1", BigDecimal.ONE));
    repository.save(new CustomerOrder("A-1", BigDecimal.TWO));
    // no error yet: both INSERTs wait in the session

    assertThatThrownBy(repository::flush)
        .isInstanceOf(DataIntegrityViolationException.class);
  }

  @Test
  void lazyLinesLoadBecauseTheTestHoldsATransaction() {
    repository.saveAndFlush(
        new CustomerOrder("B-1", BigDecimal.TEN)
            .addLine("BOOK", 2).addLine("PEN", 1));

    var order = repository.findByReference("B-1").orElseThrow();
    assertThat(order.getLines()).hasSize(2);   // green here …
  }
}
```

committed · OrderFlowIT (@SpringBootTest, no @Transactional) — 3 ✔

```
@Autowired OrderRepository repository;

@BeforeEach
void cleanSlate() { repository.deleteAll(); } // no rollback

@Test
void duplicateReferenceFailsOnSave() {
  repository.save(new CustomerOrder("A-1", BigDecimal.ONE));

  assertThatThrownBy(() -> repository.save(
          new CustomerOrder("A-1", BigDecimal.TWO)))
      .isInstanceOf(DataIntegrityViolationException.class);
}

@Test
void lazyLinesFailOutsideATransaction() {
  repository.save(
      new CustomerOrder("B-1", BigDecimal.TEN)
          .addLine("BOOK", 2).addLine("PEN", 1));

  var order = repository.findByReference("B-1").orElseThrow();
  assertThatThrownBy(() -> order.getLines().size()) // red here
      .isInstanceOf(LazyInitializationException.class);
}
```

## Same code, two outcomes

| | INSIDE A ROLLED-BACK TEST TRANSACTION | COMMITTED, AS IN PRODUCTION |
|---|---|---|
| **Duplicate `reference`** | `save()` succeeds; the error appears only at `flush()` — and a test that never flushes stays green | `save()` commits and throws `DataIntegrityViolationException` |
| **Lazy `lines` collection** | `getLines()` loads, because the test's transaction is still open | `LazyInitializationException` — the real behaviour with `open-in-view=false` |
| **Data left behind** | none — rolled back after each test | rows stay: clean up in `@BeforeEach` |

PostgreSQL's message in both runs: `ERROR: duplicate key value violates unique constraint "orders_reference_key"`. Spring translates it to `DataIntegrityViolationException`. Every test above passes — each one asserts what really happens.

## The fix the committed test protects

OrderService.java — map to plain data inside the transaction

```
/** Map to a view while the transaction is still open. */
@Transactional(readOnly = true)
public Optional<OrderView> find(String reference) {
  return repository.findWithLines(reference).map(OrderView::of);
}
```

- **No `@Transactional` on `@SpringBootTest` tests.** Let each call commit as it would in production; clean up in `@BeforeEach`.
- **In `@DataJpaTest`, flush.** Testing a constraint or a mapping? Call `flush()` or `saveAndFlush()` so the SQL actually runs.
- **Turn off open-in-view** (`spring.jpa.open-in-view=false`) so lazy loading can't silently happen while JSON is written.
- **Return DTOs, not entities,** from transactional services — then no lazy proxy can escape.
- Keep the fast rolled-back tests; add **one committed-path test** per repository or flow.

real run · Spring Boot 4.1.1 · GitHub Actions

```
✔ OrderRepositoryIT  2 tests  11.51 s  (starts PostgreSQL)
✔ OrderFlowIT        3 tests  2.471 s

WARN  org.hibernate.orm.jdbc.error : ERROR: duplicate key value
violates unique constraint "orders_reference_key"
```

**GOOD TO KNOW**

Rolled-back tests are still the right default for queries, mappings and repository methods — they are fast and leave nothing behind. Call `flush()` and `clear()` on the `EntityManager` (or `TestEntityManager`) before reading back, so the data really comes from the database.

04 · INTEGRATION & API TESTING

# Database migrations: test them on real data

A migration that works on an empty database can still fail on production's rows. Run every migration in tests against the **same database engine**, and for risky ones, against data written by the **previous** version.

src/main/resources/db/migration — Flyway 13.10

```
-- V1__create_orders.sql
CREATE TABLE orders (
  id    text PRIMARY KEY,
  email text NOT NULL,
  total numeric(12,2) NOT NULL
);

-- V2__add_status.sql
-- new column; existing rows get NULL
ALTER TABLE orders ADD COLUMN status text;

-- V3__status_required.sql
-- backfill first, THEN make it required
UPDATE orders SET status = 'PLACED' WHERE status IS NULL;
ALTER TABLE orders ALTER COLUMN status SET NOT NULL;
```

the draft someone wrote first — no backfill

```
-- V3__status_required.sql (first draft)
ALTER TABLE orders ALTER COLUMN status SET NOT NULL;
```

MigrationsIT — real PostgreSQL 17  3 ✔ · 5.866 s

```
@Test
void allMigrationsApplyToAnEmptyDatabase() {
  var result = flyway("fresh").load().migrate();

  assertThat(result.migrationsExecuted).isEqualTo(3);
  assertThat(result.targetSchemaVersion).isEqualTo("3");
  flyway("fresh").load().validate();  // checksums unchanged
}

@Test
void v3BackfillsRowsThatAlreadyExist() throws SQLException {
  flyway("live").target("2").load().migrate();   // yesterday
  insertOldOrder("live");

  flyway("live").load().migrate();               // deploy V3

  try (var c = connect(); var rs = c.createStatement()
      .executeQuery("SELECT status FROM live.orders")) {
    rs.next();
    assertThat(rs.getString(1)).isEqualTo("PLACED");
  }
}

@Test
void v3WithoutBackfillFailsOnRealData() throws SQLException {
  flyway("broken").target("2").load().migrate();
  insertOldOrder("broken");

  var withBrokenV3 = flyway("broken")
      .locations("classpath:db/broken")       // only V3
      .ignoreMigrationPatterns("*:missing")   // V1, V2 ran
      .load();

  assertThatThrownBy(withBrokenV3::migrate)
      .isInstanceOf(FlywayException.class)
      .hasMessageContaining("contains null values");
}
}
```

- **Empty database:** all three migrations apply, and `validate()` confirms no applied file was edited afterwards.
- **Yesterday's data:** migrate to V2 with `target("2")`, insert a row as the old app would, then deploy V3. The backfill sets `PLACED`.
- **The draft without backfill fails** on that row, with PostgreSQL's own error — in CI, not during the release.
- Let Hibernate **validate** the schema (`spring.jpa.hibernate.ddl-auto=validate`) instead of creating it, so entity/column mismatches fail at startup.
- Never edit an applied migration — add a new one. Flyway's checksums catch edits.
- Same idea with Liquibase: `update` to a tag, insert data, then `update` again.

## Changing a live schema without downtime: expand, then contract

| CHANGE | RELEASE 1 (EXPAND) | RELEASE 2 | RELEASE 3 (CONTRACT) |
|---|---|---|---|
| **Make a column required** | add it nullable; app writes it | backfill old rows, then `SET NOT NULL` — our V2 → V3 | — |
| **Rename a column** | add the new column; app writes both | backfill; app reads the new one | drop the old column |
| **Drop a column** | app stops reading and writing it | drop it | — |
| **Index a big table** | PostgreSQL `CREATE INDEX CONCURRENTLY`, which can't run in a transaction: set `executeInTransaction=false` in the script's `.conf` file | — | — |

During a rolling deploy the old and the new app run against the same schema, so every step must work with both. Test each step the way `MigrationsIT` does: migrate to the previous version, write data like the old app, then apply the next step.

04 · INTEGRATION & API TESTING

# Caches: Redis and `@Cacheable`

A cache is a second copy of the truth. Test what can go wrong: the second call must not reach the source, a change must evict, entries must expire — against a **real Redis**, where the serializer and the expiry live.

PriceService.java

```java
@Service
public class PriceService {
  private final PriceSource source;

  public PriceService(PriceSource source) {
    this.source = source;
  }

  @Cacheable("prices")
  public Price price(String sku) {
    return source.fetch(sku);
  }

  @CacheEvict("prices")
  public void priceChanged(String sku) {
  }

  /** Calls price() on "this", not the proxy: never cached. */
  public long priceWithTax(String sku) {
    return price(sku).paise() * 118 / 100;
  }

  @Cacheable("quotes")
  public Quote quote(String sku) {
    return new Quote(sku, source.fetch(sku).paise());
  }
}
```

RedisContainer.java — Redis 8 in Docker

```java
@TestConfiguration(proxyBeanMethods = false)
public class RedisContainer {
  @Bean
  @ServiceConnection(name = "redis")
  GenericContainer<?> redis() {
    return new GenericContainer<>("redis:8-alpine")
        .withExposedPorts(6379);
  }
}
```

PriceCacheIT — real Redis  4 ✔ · 1.253 s

```java
@SpringBootTest
@Import(RedisContainer.class)
class PriceCacheIT {
  @Autowired PriceService prices;
  @Autowired StringRedisTemplate redis;
  @MockitoSpyBean PriceSource source;

  @BeforeEach
  void emptyCache() {
    redis.delete(redis.keys("*"));
    clearInvocations(source);
  }

  @Test
  void secondCallIsServedFromRedis() {
    prices.price("book-1");
    assertThat(redis.hasKey("prices::book-1")).isTrue();
    prices.price("book-1");

    verify(source, times(1)).fetch("book-1");
    assertThat(redis.hasKey("prices::book-1")).isTrue();
    assertThat(redis.getExpire("prices::book-1"))
        .isBetween(1L, 600L);              // TTL 10m applied
  }

  @Test
  void evictingMakesTheNextCallFetchAgain() {
    prices.price("book-1");
    prices.priceChanged("book-1");
    prices.price("book-1");

    verify(source, times(2)).fetch("book-1");
  }

  @Test
  void selfInvocationBypassesTheCache() {
    prices.priceWithTax("book-1");
    prices.priceWithTax("book-1");

    verify(source, times(2)).fetch("book-1");
    assertThat(redis.hasKey("prices::book-1")).isFalse();
  }

  @Test
  void aValueThatIsNotSerializableFailsOnlyAtRuntime() {
    assertThatThrownBy(() -> prices.quote("book-1"))
        .isInstanceOf(IllegalStateException.class)
        .hasMessage("Cannot serialize value of type "
            + "shop.cache.Quote without a serializer");
  }
}
```

## Five cache bugs the tests catch

| BUG | WHAT THE TEST SHOWED |
|---|---|
| **Cache never used** | two calls, `fetch()` ran once; the key `prices::book-1` is in Redis |
| **Stale after a change** | after `@CacheEvict` the next call fetches again |
| **Self-invocation** | `priceWithTax()` calls `price()` on `this`, not on the proxy: fetched twice, nothing in Redis |
| **Value can't be stored** | `Quote` isn't `Serializable`: `IllegalStateException: Cannot serialize value of type shop.cache.Quote without a serializer` — on the first put, at runtime |
| **Never expires** | `getExpire()` is between 1 and 600 s: `time-to-live=10m` reached Redis |

**RULE OF THUMB**

Tests that are not about caching set `spring.cache.type=none`, so they can't pass by reading a value an earlier test left behind.

**GOOD TO KNOW**

Our two Redis test classes use different Spring configurations, so each got its own context — and its own container (started in 0.2 s and 0.2 s). Same configuration, one context, one container.

04 · INTEGRATION & API TESTING

# A flaky cache test: Redis writes are asynchronous

Our first `PriceCacheIT` failed now and then. Repeated in CI it failed **6 times in 30 runs**: sometimes the value was not in Redis right after `price()` returned; sometimes the call after `@CacheEvict` still got the old value. It was not a test bug. Since Spring Data Redis 4.0, with Lettuce, `put`, `evict` and `clear` are **asynchronous**: they "may complete after the calling method returns".

AsyncCacheWritesIT — the default, measured — 1 ✔ · 2.234 s

```
/** The library default, no immediate writes: count the lag. */
@SpringBootTest(properties =
    "shop.cache.immediate-writes=false")
@Import(RedisContainer.class)
class AsyncCacheWritesIT {
  @Autowired PriceService prices;
  @Autowired StringRedisTemplate redis;

  @Test
  void writesLandAfterTheCallReturns() {
    int notYet = 0;
    for (int i = 0; i < 1000; i++) {
      prices.price("sku-" + i);
      if (!redis.hasKey("prices::sku-" + i)) {
        notYet++;
      }
    }
    System.out.println("[async writes] " + notYet + " of 1000"
        + " entries were not in Redis when price() returned");

    await().atMost(Duration.ofSeconds(5)).until(
        () -> redis.keys("prices::sku-*").size() == 1000);
  }
}
```

real output · CI

```
[async writes] 303 of 1000 entries were not in Redis when
price() returned
```

CacheConfig.java — the fix: wait for writes

```
/**
 * Spring Data Redis 4 with Lettuce: put() and evict() are
 * asynchronous by default and can finish after the call
 * returns. Wait for them: a read right after a write must
 * see it.
 */
@Configuration(proxyBeanMethods = false)
class CacheConfig {
  @Bean
  @ConditionalOnProperty(name = "shop.cache.immediate-writes",
      havingValue = "true", matchIfMissing = true)
  RedisCacheManagerBuilderCustomizer immediateWrites(
      RedisConnectionFactory redis) {
    return builder -> builder.cacheWriter(RedisCacheWriter
        .create(redis, w -> w.immediateWrites()));
  }
}
```

## What the failing runs reported

| TEST | WHAT IT SAW | WHY |
|---|---|---|
| `secondCallIsServedFromRedis` | right after `price()` the key was not in Redis: `Expecting value to be true but was false` | the `SET` was still on its way |
| `evictingMakesTheNextCallFetchAgain` | after `priceChanged()` the next call still got the cached value: `Wanted 2 times … But was 1 time` | the `DEL` was still on its way |

Source: Spring Data Redis reference, "Migration Guides", 3.x → 4.x: the default `RedisCacheWriter` is asynchronous for `put`, `evict` and `clear` when the connection factory is also a `ReactiveRedisConnectionFactory` — which Lettuce's is. There is no Spring Boot property for it; the customizer above is the switch.

## How we found it: repeat the test

flake-hunt.yml — run from the Actions tab

```
for i in $(seq "$RUNS"); do
  if ! mvn -B -ntp verify -Dit.test="$TEST" -Dtest=NoUnit \
    -Dsurefire.failIfNoSpecifiedTests=false > run.log 2>&1; then
    fails=$((fails+1))
    echo "::warning title=run $i failed::$msg"
  fi
done
```

Three machines, ten runs each: 3, 2 and 1 failures before the fix; 0, 0 and 0 after it.

- **A flaky test is a bug report** — about the test or about the code. Find out which before you add a retry.
- **Measure the window:** 303 of 1000 writes had not landed when the method returned. The same race exists in production: an evicted price can be served once more.
- **Decide on purpose.** Need read-your-writes (evict on change, then read)? Use `immediateWrites()`. Prefer throughput? Keep the default and let tests wait with Awaitility, as `AsyncCacheWritesIT` does.
- `Cache.evictIfPresent(…)` and `invalidate(…)` always wait, whatever the setting.
- **A retry would have hidden it.** Surefire's `rerunFailingTestsCount` turns this into a "flaky pass" — and the production race stays.
- With the fix, the strict test stayed green in 30 of 30 repeated runs.

04 · INTEGRATION & API TESTING

# Scheduled jobs: the rule and the wiring

A scheduled job is two things: a **rule** (which carts are abandoned?) and **wiring** (does it really run every ten minutes?). Test the rule as a plain method with a fixed clock — fast, exact, every boundary. Test the wiring once, with the schedule shortened and Awaitility waiting for the effect.

CartCleanup.java

```java
@Component
public class CartCleanup {
  static final Duration MAX_AGE = Duration.ofHours(24);
  private final CartStore carts;
  private final Clock clock;

  public CartCleanup(CartStore carts, Clock clock) {
    this.carts = carts;
    this.clock = clock;
  }

  /** The schedule is wiring; the rule is a plain method. */
  @Scheduled(fixedDelayString = "${shop.cleanup.every:PT10M}")
  public void scheduledRun() {
    removeAbandoned();
  }

  public int removeAbandoned() {
    var cutoff = clock.instant().minus(MAX_AGE);
    return carts.removeTouchedBefore(cutoff);
  }
}
```

CartCleanupTest — unit, fixed clock  2 ✔ · 0.018 s

```java
/** The rule, without Spring or a scheduler: plain and fast. */
class CartCleanupTest {
  Instant now = Instant.parse("2026-10-13T10:00:00Z");
  CartStore carts = new CartStore();
  CartCleanup cleanup = new CartCleanup(carts,
      Clock.fixed(now, ZoneOffset.UTC));

  @Test
  void removesCartsUntouchedForMoreThan24Hours() {
    carts.touch("old", now.minus(Duration.ofHours(25)));
    carts.touch("fresh", now.minus(Duration.ofHours(1)));

    assertThat(cleanup.removeAbandoned()).isEqualTo(1);
    assertThat(carts.ids()).containsExactly("fresh");
  }

  @Test
  void exactly24HoursOldIsKept() {
    carts.touch("edge", now.minus(Duration.ofHours(24)));

    assertThat(cleanup.removeAbandoned()).isZero();
  }
}
```

CartCleanupScheduleIT — real scheduler  1 ✔ · 0.942 s

```java
/** The wiring: the schedule really fires (here every 0.2 s). */
@SpringBootTest(properties = {"shop.cleanup.every=PT0.2S",
    "spring.cache.type=none"})
class CartCleanupScheduleIT {
  @Autowired CartStore carts;

  @Test
  void theScheduledRunRemovesAbandonedCarts() {
    carts.touch("abandoned", Instant.EPOCH);

    await().atMost(Duration.ofSeconds(5))
        .until(() -> carts.ids().isEmpty());
  }
}
```

NightlyReport.java — a cron schedule

```java
/** 02:00 on weekdays, India time, whatever the server's zone. */
public static final String CRON = "0 0 2 * * MON-FRI";
public static final String ZONE = "Asia/Kolkata";
private final AtomicInteger runs = new AtomicInteger();

@Scheduled(cron = CRON, zone = ZONE)
```

NightlyReportCronTest  2 ✔ · 0.013 s

```java
/** A cron expression is code: test when it fires. */
class NightlyReportCronTest {
  CronExpression cron = CronExpression.parse(NightlyReport.CRON);

  @Test
  void fridayNightRunsNextOnMondayAt2() {
    var friday = LocalDateTime.parse("2026-10-16T22:00");
    assertThat(cron.next(friday))
        .isEqualTo(LocalDateTime.parse("2026-10-19T02:00"));
  }

  @Test
  void exactly2amRunsTheNextWeekdayNotAgain() {
    var tuesday2am = LocalDateTime.parse("2026-10-13T02:00");
    assertThat(cron.next(tuesday2am))
        .isEqualTo(LocalDateTime.parse("2026-10-14T02:00"));
  }
}
```

- **Inject a `Clock`**, never call `Instant.now()`: the unit test pins time and tests "exactly 24 hours" both ways.
- **Keep the rule a plain public method.** The `@Scheduled` method only calls it, so the rule needs no Spring.
- **The schedule is a property** (`${shop.cleanup.every:PT10M}`), so one test can run it every 0.2 s.
- **A cron expression is code.** `CronExpression.parse(…).next(…)` answers "when next?" without waiting; pin the `zone`.
- **Wait for the effect** with `await().atMost(…)`; never `Thread.sleep` and hope.
- A `fixedDelay` job also runs once as soon as the context starts — in every `@SpringBootTest`. If it changes shared data, put it behind `@ConditionalOnProperty` and switch it off in tests.

04 · INTEGRATION & API TESTING

# Batch jobs: Spring Batch 6

A batch job reads, processes and writes in chunks, and must decide what to do with bad records. Test the processor as plain Java; test the whole job with `@SpringBatchTest` on a **small file in which every line has a purpose** — then the counts prove the job did the right thing with each one.

ImportPricesJob.java — the step

```java
@Bean
Step importStep(JobRepository repo, PriceCatalog catalog,
    FlatFileItemReader<PriceLine> reader) {
  return new StepBuilder("importStep", repo)
      .<PriceLine, Price>chunk(100)
      .reader(reader)
      .processor(new ToPaise())
      .writer(chunk -> chunk.forEach(catalog::put))
      .faultTolerant()
      .skip(FlatFileParseException.class)
      .skipLimit(5)
      .build();
}
```

ToPaise.java — the processor

```java
/** Rupees to paise; a negative price is filtered (null). */
public class ToPaise
    implements ItemProcessor<PriceLine, Price> {
  @Override
  public Price process(PriceLine line) {
    if (line.rupees().signum() < 0) {
      return null;
    }
    BigDecimal paise = line.rupees().movePointRight(2);
    return new Price(line.sku(), paise.longValueExact());
  }
}
```

ImportPricesJobIT — the whole job  1 ✔ · 0.539 s

```java
/** prices.csv: 3 good lines, 1 negative, 1 broken. */
@SpringBootTest(properties = "spring.cache.type=none")
@SpringBatchTest
class ImportPricesJobIT {
  @Autowired JobOperatorTestUtils jobs;
  @Autowired PriceCatalog catalog;

  @Test
  void filtersNegativeAndSkipsBrokenLines() throws Exception {
    JobExecution run = jobs.startJob();

    assertThat(run.getStatus())
        .isEqualTo(BatchStatus.COMPLETED);
    StepExecution step =
        run.getStepExecutions().iterator().next();
    assertThat(step.getReadCount()).isEqualTo(4);
    assertThat(step.getFilterCount()).isEqualTo(1);
    assertThat(step.getReadSkipCount()).isEqualTo(1);
    assertThat(step.getWriteCount()).isEqualTo(3);
    assertThat(catalog.get("book-2"))
        .contains(new Price("book-2", 129_950));
  }
}
```

## prices.csv: five lines, five cases (line 1 is the header, skipped by `linesToSkip(1)`)

| LINE | CONTENT | WHAT THE JOB DID | COUNTED IN |
|---|---|---|---|
| **2** | book-1,499.00 | read, processed, **written**: 49,900 paise | write count |
| **3** | book-2,1299.50 | read, processed, **written**: 129,950 paise | write count |
| **4** | pen-1,-5.00 | read; the processor returns `null`: **filtered** | filter count |
| **5** | book-4 | one field instead of two: `FlatFileParseException`, **skipped** | read skip count |
| **6** | book-5,250 | read, processed, **written**: 25,000 paise | write count |

ToPaiseTest — no Batch needed  2 ✔ · 0.006 s

```java
/** The processor is plain Java: test it without Batch. */
class ToPaiseTest {
  ToPaise toPaise = new ToPaise();

  @Test
  void convertsRupeesToPaise() {
    var line = new PriceLine("book-2",
        new BigDecimal("1299.50"));
    assertThat(toPaise.process(line))
        .isEqualTo(new Price("book-2", 129_950));
  }

  @Test
  void filtersNegativePrices() {
    var line = new PriceLine("pen-1", new BigDecimal("-5.00"));
    assertThat(toPaise.process(line)).isNull();
  }
}
```

- The job **completed**: read 4, filtered 1, skipped 1, wrote 3 — the step ran in 21 ms in CI.
- `JobOperatorTestUtils` (Batch 6; formerly `JobLauncherTestUtils`) starts the job with unique parameters, so it can run again.
- `spring.batch.job.enabled=false`: Boot otherwise runs the job when the application starts.
- No database needed: with `spring-boot-starter-batch` the job repository is in memory; `spring-boot-starter-batch-jdbc` stores it in tables.
- Batch 6 builds a step with `chunk(100)`; the transaction manager is optional (the `chunk(int, PlatformTransactionManager)` form is deprecated).
- Also test the **skip limit**: one more bad line than allowed must fail the job, not import half a file.

04 · INTEGRATION & API TESTING

# Messaging & async: Kafka with Testcontainers

Kafka (like most brokers) delivers **at least once**: after a consumer crash or rebalance, a message can arrive again. So the test that matters is not “can I send and receive” but “does a **duplicate** do any harm?”. A real broker in a container answers that.

OrderEventConsumer.java — idempotent by event id

```java
/**
 * Kafka delivers at least once: an event can arrive twice.
 * This consumer is idempotent - each event id is applied once.
 */
public class OrderEventConsumer implements AutoCloseable {
  private final KafkaConsumer<String, String> consumer;
  private final Set<String> applied =
      ConcurrentHashMap.newKeySet();
  private final AtomicInteger received = new AtomicInteger();
  private final Thread loop;

  public OrderEventConsumer(String servers, String topic) {
    consumer = new KafkaConsumer<>(Map.of(
        ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, servers,
        ConsumerConfig.GROUP_ID_CONFIG, "order-projection",
        ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest"),
        new StringDeserializer(), new StringDeserializer());
    consumer.subscribe(List.of(topic));
    loop = Thread.ofPlatform().name("order-events")
        .start(this::run);
  }

  private void run() {
    try {
      while (true) {
        var records = consumer.poll(Duration.ofMillis(200));
        for (var record : records) {
          received.incrementAndGet();
          var id = record.headers().lastHeader("event-id");
          applied.add(new String(id.value(), UTF_8));
          // adding an id that is already there changes nothing
        }
      }
    } catch (WakeupException stopping) {
      // close() was called
    } finally {
      consumer.close();
    }
  }

  public int received() { return received.get(); }
  public int applied() { return applied.size(); }

  @Override public void close() throws InterruptedException {
    consumer.wakeup();
    loop.join();
  }
}
```

OrderEventsIT — real Kafka 4.3.1 broker — 1 ✔ · 3.961 s

```java
class OrderEventsIT {
  static final KafkaContainer KAFKA =
      new KafkaContainer("apache/kafka-native:4.3.1");
  static { KAFKA.start(); }      // singleton per test JVM

  @Test
  void redeliveredEventIsAppliedOnlyOnce() throws Exception {
    String kafka = KAFKA.getBootstrapServers();
    String paid = "{\"status\":\"PAID\"}";
    try (var out = new OrderEventPublisher(kafka, "orders");
         var in = new OrderEventConsumer(kafka, "orders")) {

      out.publish("evt-1", "ord-1", paid);
      out.publish("evt-1", "ord-1", paid);    // redelivered
      out.publish("evt-2", "ord-2", paid);

      await().atMost(Duration.ofSeconds(30))  // no sleep
          .untilAsserted(() -> {
            assertThat(in.received()).isEqualTo(3);
            assertThat(in.applied()).isEqualTo(2);
          });
    }
  }
}
```

OrderEventPublisher.java (excerpt)

```java
  public void publish(String eventId, String orderId,
      String json)
      throws Exception {
    var record = new ProducerRecord<>(topic, orderId, json);
    record.headers().add("event-id",
        eventId.getBytes(StandardCharsets.UTF_8));
    producer.send(record).get();      // wait for the broker ack
  }
```

- **Real run:** the `apache/kafka-native` container started in 0.5 s; three messages arrived, two were applied.
- **Wait, don’t sleep.** Awaitility polls until the assertion holds or the timeout expires — fast when things work, a clear failure when they don’t.
- **Isolate tests:** a unique topic or consumer group per test, so leftovers from one test can’t satisfy another.
- **Test the unhappy paths:** a poison message, a consumer that crashes mid-batch, a redelivery after restart.

| OPTION | WHAT IT IS | USE FOR |
|---|---|---|
| **Kafka in Testcontainers** | the real broker image, one per JVM | the default |
| `@EmbeddedKafka` | spring-kafka-test starts a broker inside the JVM | Spring Kafka apps |
| `MockProducer` / `MockConsumer` | kafka-clients’ in-memory test doubles | unit tests of your own logic |
| **Contract tests** | the event’s schema agreed with consumers | events crossing team lines |

04 · INTEGRATION & API TESTING

# E-mail: testing with GreenMail

Customers notice e-mails more than most features: a wrong total, a missing confirmation, a mail sent twice. **GreenMail** (Apache-2.0) runs a real SMTP server inside the test JVM, so your code sends real mail over a real socket — and the test reads it back.

OrderMailer.java — plain Jakarta Mail (Angus Mail 2.0.5)

```java
/** Sends order e-mails through any SMTP server. */
public class OrderMailer {
  private final Session session;

  public OrderMailer(String host, int port) {
    Properties props = new Properties();
    props.put("mail.smtp.host", host);
    props.put("mail.smtp.port", String.valueOf(port));
    props.put("mail.smtp.connectiontimeout", "2000");
    session = Session.getInstance(props);
  }

  public void sendConfirmation(String to, String orderId,
      String total) throws MessagingException {
    var message = new MimeMessage(session);
    message.setFrom(new InternetAddress("orders@shop.example"));
    message.setRecipients(Message.RecipientType.TO, to);
    message.setSubject("Order " + orderId + " confirmed",
        "UTF-8");
    message.setText("Thank you! Total: " + total, "UTF-8");
    Transport.send(message);
  }
}
```

OrderMailerTest — GreenMail JUnit 5 extension — 2 ✔ · 0.158 s

```java
/** GreenMail: a real SMTP server inside the test JVM. */
class OrderMailerTest {
  @RegisterExtension                  // fresh mailbox per test
  static GreenMailExtension smtp = new GreenMailExtension(
      ServerSetupTest.SMTP.dynamicPort());   // any free port

  OrderMailer mailer() {
    int port = smtp.getSmtp().getPort();
    return new OrderMailer("localhost", port);
  }

  @Test
  void confirmationIsSentToTheCustomer() throws Exception {
    mailer().sendConfirmation("asha@example.com", "ord-1",
        "₹1,140.00");

    assertThat(smtp.waitForIncomingEmail(2_000, 1)).isTrue();
    MimeMessage mail = smtp.getReceivedMessages()[0];
    assertThat(mail.getSubject())
        .isEqualTo("Order ord-1 confirmed");
    assertThat(mail.getAllRecipients())
        .extracting(Object::toString)
        .containsExactly("asha@example.com");
    assertThat((String) mail.getContent())
        .isEqualTo("Thank you! Total: ₹1,140.00");
  }

  @Test
  void smtpServerDownIsAnError() {
    var nobodyListening = new OrderMailer("localhost", 1);

    assertThatThrownBy(() -> nobodyListening.sendConfirmation(
            "asha@example.com", "ord-2", "₹10.00"))
        .isInstanceOf(MessagingException.class);
  }
}
```

| CHECK IN EVERY E-MAIL TEST | WHY |
|---|---|
| **Recipient and sender** | a wrong address is a privacy incident |
| **Subject and body text** | order number, total, links — what the customer reads |
| **Encoding** | the ₹ sign survived here because subject and body are sent as UTF-8 |
| **Failure path** | server down or rejected: retry, queue, or tell the user |

## Four ways to test e-mail

| OPTION | WHAT IT IS | USE FOR |
|---|---|---|
| **Mock the port** | Mockito mock of your `EmailSender` interface (p.14) | unit tests: was an e-mail requested? |
| **GreenMail in-process** | `greenmail-junit5` 2.1.14: real SMTP, IMAP, POP3 inside the test JVM | the default: real SMTP, no Docker |
| **GreenMail in a container** | the `greenmail/standalone` image | non-Java services, shared test environments |
| **Mailpit** | SMTP server with a web UI and REST API, in a container | end-to-end and manual checks of real HTML mails |

- **Assert what the customer sees:** recipient, subject, body — and that the server being down is an error you handle, not a lost mail.
- **Free ports:** `ServerSetupTest.SMTP` is a fixed port (3025); `.dynamicPort()` picks a free one, so parallel forks never clash.
- **Spring Boot:** point `spring.mail.host` and `spring.mail.port` at GreenMail with `@DynamicPropertySource`; `JavaMailSender` needs no change.
- Never let tests reach a real mail server: one wrong address and a customer gets a test mail.

04 · INTEGRATION & API TESTING

# Cloud services: AWS and Azure emulators

Code that talks to S3, SQS or Azure Storage can be tested against **emulators** in containers: the real SDK, real HTTP, no cloud account and no bill. One abstract test class runs the same checks against every AWS emulator; Azure gets the same two tests.

AwsStorageContract — AWS SDK v2 (excerpt)

```
  @Test
  void storesAndReadsAnInvoice() {
    try (var s3 = s3()) {
      s3.createBucket(b → b.bucket("invoices"));
      s3.putObject(b → b.bucket("invoices").key("ord-1.txt"),
          RequestBody.fromString("TOTAL 1768.74"));

      String text = s3.getObjectAsBytes(b → b
          .bucket("invoices").key("ord-1.txt")).asUtf8String();
      assertThat(text).isEqualTo("TOTAL 1768.74");
    }
  }

  @Test
  void queuesAndReceivesAMessage() {
    try (var sqs = sqs()) {
      String url = sqs.createQueue(b → b.queueName("orders"))
          .queueUrl();
      sqs.sendMessage(b → b.queueUrl(url)
          .messageBody("ord-1 PAID"));

      var messages = sqs.receiveMessage(
          b → b.queueUrl(url).waitTimeSeconds(5)).messages();
      assertThat(messages).extracting(Message::body)
          .containsExactly("ord-1 PAID");
    }
  }
  // queuesAndReceivesAMessage(): SQS create / send / receive
}
```

MotoAwsIT — free, open source — 2 ✔ · 10.44 s

```
class MotoAwsIT extends AwsStorageContract {
  static final GenericContainer<?> MOTO =
      new GenericContainer<>("motoserver/moto:5.2.4")
          .withExposedPorts(5000);
  static { MOTO.start(); }

  @Override URI endpoint() {
    return URI.create("http://" + MOTO.getHost() + ":"
        + MOTO.getMappedPort(5000));
  }
}
```

LocalStackAwsIT — only with a token — 2 tests, 2 skipped

```
@EnabledIfEnvironmentVariable(named = "LOCALSTACK_AUTH_TOKEN",
                              matches = ".+")
class LocalStackAwsIT extends AwsStorageContract {
  // … same tests, LocalStack endpoint
}
```

AzureStorageIT — Azure SDK + Azurite — 2 ✔ · 19.55 s

```
class AzureStorageIT {
  static final AzuriteContainer AZURITE =
      new AzuriteContainer(
          "mcr.microsoft.com/azure-storage/azurite:3.37.0");
  static { AZURITE.start(); }

  @Test
  void storesAndReadsAnInvoice() {
    var blobs = new BlobServiceClientBuilder()
        .connectionString(AZURITE.getConnectionString())
        .buildClient();
    var invoices = blobs.createBlobContainer("invoices");
    invoices.getBlobClient("ord-1.txt")
        .upload(BinaryData.fromString("TOTAL 1768.74"));

    String text = invoices.getBlobClient("ord-1.txt")
        .downloadContent().toString();
    assertThat(text).isEqualTo("TOTAL 1768.74");
  }

  // queuesAndReceivesAMessage(): QueueClient send / receive
}
```

| CLOUD | EMULATOR | COVERS | NOTES |
|---|---|---|---|
| **AWS** | **Moto** (`motoserver/moto`) | S3, SQS, DynamoDB, SNS and many more | Apache-2.0; no account. Ran here. |
| **AWS** | **LocalStack** | broad AWS coverage, high fidelity | Since 23 Mar 2026 needs a LocalStack account and `LOCALSTACK_AUTH_TOKEN`, also in CI. Skipped here. |
| **Azure** | **Azurite** | Blob, Queue, Table storage | Microsoft, MIT licence. Ran here. |
| **Azure** | Service Bus / Event Hubs emulators | messaging | Testcontainers modules; you must `acceptLicense()`; Service Bus needs SQL Server, Event Hubs needs Azurite |
| **Azure** | Cosmos DB emulator | NoSQL database | Testcontainers module; Linux image |

Container start-up in our run: Moto 1.1 s, Azurite 1.8 s. Emulators are not the cloud: IAM policies, quotas, latency and some API details differ — keep a few smoke tests against a real account.

04 · INTEGRATION & API TESTING

# Secured APIs: testing with a mock auth server

An OAuth2 resource server checks every token's **signature, issuer, audience and scope**. Tests need tokens that pass — and tokens that must fail. Pick the lightest level that exercises what you are checking:

| LEVEL | HOW | TESTS | SPEED |
|---|---|---|---|
| **1 · No server** | `jwt()` from spring-security-test builds the token in memory | controller tests: 401, 403, 200 | milliseconds |
| **2 · In-process server** | `MockOAuth2Server` as a library: real signed tokens, real JWKS, no Docker | full-app tests of issuer, audience, signature | milliseconds |
| **3 · Server in a container** | the same server as an image; tokens fetched over HTTP (client_credentials) | the real token flow end to end | seconds |
| **4 · Keycloak** | a full identity provider via Testcontainers (community module) | login flows, realms, roles as configured in prod | slowest |

1 · @WebMvcTest + jwt() — 3 ✔ · 2.665 s

```
@Test
void tokenWithoutTheScopeIs403() {
  assertThat(mvc.get().uri("/reports/daily").with(jwt()))
      .hasStatus(HttpStatus.FORBIDDEN);
}

@Test
void tokenWithTheScopeIs200() {
  var token = jwt().jwt(t -> t.subject("asha")).authorities(
      new SimpleGrantedAuthority("SCOPE_reports.read"));
  assertThat(mvc.get().uri("/reports/daily").with(token))
      .hasStatusOk()
      .bodyJson().isLenientlyEqualTo("""
          {"orders":42,"requestedBy":"asha"}""");
}
}
```

2 · in-process server (lightweight) — 2 ✔ · 1.149 s

```
class InProcessAuthServerIT {
  static final MockOAuth2Server AUTH = new MockOAuth2Server();
  static { AUTH.start(); }

  @DynamicPropertySource
  static void authServer(DynamicPropertyRegistry props) {
    props.add(
        "spring.security.oauth2.resourceserver.jwt.issuer-uri",
        () -> AUTH.issuerUrl("shop").toString());
  }

  @AfterAll static void stop() { AUTH.shutdown(); }

  // report(token): GET /reports/daily with the bearer token

  @Test
  void tokenWithScopeIsAccepted() {
    var scope = Map.<String, Object>of("scope", "reports.read");
    String token = AUTH.issueToken("shop", "asha",
        "reports-api", scope).serialize();

    assertThat(report(token)).contains("asha");
  }
```

3 · server in a container — 3 ✔ · 10.36 s

```
static final GenericContainer<?> AUTH =
    new GenericContainer<>(
        "ghcr.io/navikt/mock-oauth2-server:6.0.7")
        .withExposedPorts(8080)
        .withEnv("JSON_CONFIG", """
            {"tokenCallbacks": [{"issuerId": "shop",
              "requestMappings": [{"requestParam": "scope",
                "match": "reports.read",
                "claims": {"sub": "reporting-job",
                  "aud": ["reports-api"],
                  "scope": "reports.read"}}]
            }]}""");
static { AUTH.start(); }
```

- **Always test the failures:** no token (401), wrong scope (403), another issuer or audience (401). A filter chain that lets everything through passes every happy-path test.
- `mock-oauth2-server` 6.0.7 (navikt, MIT) puts client_credentials scopes in `aud` by default — the JSON config above adds a `scope` claim.

3 · fetching a real token over HTTP

```
/** client_credentials grant against the mock server. */
String token(String issuerId, String scope) {
  var form = new LinkedMultiValueMap<String, String>();
  form.add("grant_type", "client_credentials");
  form.add("client_id", "reporting-job");
  form.add("client_secret", "any");
  form.add("scope", scope);
  Map<?, ?> body = http.post()
      .uri(issuer(issuerId) + "/token")
      .contentType(MediaType.APPLICATION_FORM_URLENCODED)
      .body(form).retrieve().body(Map.class);
  return (String) body.get("access_token");
}
```

real run · Spring Boot 4.1.1 · GitHub Actions

```
✔ ReportControllerTest   3 tests  2.665 s
✔ InProcessAuthServerIT  2 tests  1.149 s
✔ MockAuthServerIT       3 tests  10.36 s

mock-oauth2-server container started in 1.7 s
Spring context for MockAuthServerIT: 1.598 s
Spring context for InProcessAuthServerIT: 0.361 s
```

04 · INTEGRATION & API TESTING

# Calling other services: WireMock & retries

Your code will meet slow, failing and strange responses from the services it calls. **WireMock** (Apache-2.0) is a real HTTP server you script per test, so you can produce those failures on demand — and prove your timeouts and retries (here **Resilience4j**) behave.

PriceClient.java — timeout, error mapping (excerpt)

```java
public BigDecimal priceOf(String sku) {
  return retry.executeSupplier(() -> fetch(sku));
}

private BigDecimal fetch(String sku) {
  var request = HttpRequest.newBuilder(
          baseUrl.resolve("/prices/" + sku))
      .timeout(Duration.ofMillis(500))   // never wait forever
      .build();
  HttpResponse<String> response;
  try {
    response = http.send(request,
        HttpResponse.BodyHandlers.ofString());
  } catch (IOException e) {              // timeouts included
    throw new UpstreamException("unreachable", e);
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new IllegalStateException(e);
  }
  int status = response.statusCode();
  if (status == 404) throw new NoSuchElementException(sku);
  if (status ⩾ 500) {
    throw new UpstreamException("HTTP " + status, null);
  }
  var m = PRICE.matcher(response.body());
  if (!m.find()) {
    throw new IllegalStateException("bad price JSON: "
        + response.body());
  }
  return new BigDecimal(m.group(1));
}
}
```

Retries.java — Resilience4j 2.4.0

```java
/** 3 attempts, 100 ms apart, only for retryable failures. */
public static Retry forPrices() {
  return Retry.of("prices", RetryConfig.custom()
      .maxAttempts(3)
      .waitDuration(Duration.ofMillis(100))
      .retryExceptions(UpstreamException.class)
      .build());
}
```

PriceClientTest — WireMock 3.13.2 5 ✔ · 3.421 s

```java
/** WireMock: a real HTTP server that answers as told. */
class PriceClientTest {
  @RegisterExtension
  static WireMockExtension wm = WireMockExtension.newInstance()
      .options(wireMockConfig().dynamicPort()).build();

  static final String PATH = "/prices/BOOK";
  static final String BOOK = """
      {"sku":"BOOK","price":"600.00"}""";

  PriceClient client = new PriceClient(
      URI.create(wm.baseUrl()), Retries.forPrices());

  @Test
  void returnsThePrice() {
    wm.stubFor(get(PATH).willReturn(okJson(BOOK)));

    assertThat(client.priceOf("BOOK")).isEqualTo("600.00");
  }

  @Test
  void twoOutagesThenSuccessIsRetried() {
    wm.stubFor(get(PATH).inScenario("flaky")
        .whenScenarioStateIs(STARTED)
        .willReturn(serviceUnavailable()).willSetStateTo("2"));
    wm.stubFor(get(PATH).inScenario("flaky")
        .whenScenarioStateIs("2")
        .willReturn(serverError()).willSetStateTo("up"));
    wm.stubFor(get(PATH).inScenario("flaky")
        .whenScenarioStateIs("up").willReturn(okJson(BOOK)));

    assertThat(client.priceOf("BOOK")).isEqualTo("600.00");
    wm.verify(3, getRequestedFor(urlEqualTo(PATH)));
  }

  @Test
  void slowServiceTimesOutAfterThreeTries() {
    wm.stubFor(get(PATH)
        .willReturn(okJson(BOOK).withFixedDelay(2_000)));

    assertThatThrownBy(() -> client.priceOf("BOOK"))
        .isInstanceOf(UpstreamException.class)
        .hasMessage("unreachable");
    wm.verify(3, getRequestedFor(urlEqualTo(PATH)));
  }

  // … unknownProductIsNotRetried, malformedJsonIsAClearError
}
```

## What each stub proves

| STUB | EXPECTED BEHAVIOUR | CHECKED WITH |
|---|---|---|
| **503, then 500, then 200 (a scenario)** | two retries, then the price | `verify(3, getRequestedFor(…))` |
| **200 after a 2-second delay** | 500 ms timeout, three attempts, then `UpstreamException` | `withFixedDelay(2_000)`, 3 requests |
| **404** | **no** retry — retrying can't make a missing product appear | exactly 1 request |
| **HTML instead of JSON** | a clear error naming the bad body | `hasMessageStartingWith` |

- **Retry only what can succeed next time:** timeouts and 5xx yes; 4xx and bad data no.
- **Always set a timeout.** Java's `HttpClient` waits forever by default; one slow service then ties up every thread.
- **Verify the number of calls,** not just the result — a retry loop that silently fires 50 times is a bug too.
- Stubs drift from the real service; pair them with a contract test (p.31).

04 · INTEGRATION & API TESTING

# Contract testing with Pact

A WireMock stub says what *you think* the other service returns. A **consumer-driven contract** makes that belief checkable: the consumer's test records what it needs, and the provider's build replays it against the real API. Break the contract and the **provider's** build goes red — before anything ships.

consumer test — 1 ✔ · 0.882 s

```
@ExtendWith(PactConsumerTestExt.class)
@PactTestFor(providerName = "orders")
class OrderStatusConsumerPactTest {

  @Pact(consumer = "shop", provider = "orders")
  V4Pact paidOrder(PactDslWithProvider builder) {
    return builder
        .given("order ord-1 is paid")
        .uponReceiving("a status request for ord-1")
          .path("/orders/ord-1/status").method("GET")
        .willRespondWith()
          .status(200)
          .body(new PactDslJsonBody()
              .stringValue("id", "ord-1")
              .stringType("status", "PAID"))   // any string
        .toPact(V4Pact.class);
  }

  @Test
  void readsTheStatus(MockServer server) throws Exception {
    var client =
        new OrderStatusClient(URI.create(server.getUrl()));

    assertThat(client.statusOf("ord-1")).isEqualTo("PAID");
  }
}
```

provider test — 1 ✔ · 0.702 s

```
@Provider("orders")
@PactFolder("target/pacts")      // in real life: a Pact Broker
class OrderStatusProviderPactIT {
  static OrderStatusApi api;

  @BeforeAll static void start() throws Exception {
    api = new OrderStatusApi();
  }

  @AfterAll static void stop() { api.close(); }

  @BeforeEach void target(PactVerificationContext context) {
    context.setTarget(
        new HttpTestTarget("localhost", api.port()));
  }

  @State("order ord-1 is paid")
  void orderIsPaid() { /* set up data; ours is fixed */ }

  @TestTemplate
  @ExtendWith(PactVerificationInvocationContextProvider.class)
  void honoursTheContract(PactVerificationContext context) {
    context.verifyInteraction();
  }
}
```

the contract: target/pacts/shop-orders.json

```
"request": {
        "method": "GET",
        "path": "/orders/ord-1/status"
      },
      "response": {
        "body": {
          "content": {
            "id": "ord-1",
            "status": "PAID"
          }
  … }
```

real output · provider renamed status → state

```
Verifying a pact between shop and orders
  [Using File target/pacts/shop-orders.json]
  Given order ord-1 is paid
  a status request for ord-1
    returns a response which
      has status code 200 (OK)
      has a matching body (FAILED)
1.1) body: $ Actual map is missing the following keys: status
        {
          "id": "ord-1",
        -  "status": "PAID"
        +  "state": "PAID",
        +  "total": "1140.00"
        }
```

- **The consumer asks only for what it uses:** `id` and `status`. The provider's extra `total` field didn't matter — providers stay free to add.
- `stringType` matches any string; `stringValue` requires that exact value. Match on shape unless the value matters.
- **In real life** contracts travel through a **Pact Broker**, and `can-i-deploy` blocks a release whose contracts aren't verified.
- Pact JVM 4.7.5 on GitHub Actions: green on the current API, red once the field was renamed. `@State` methods set up each interaction's data.
- Spring Cloud Contract works the other way round: the provider writes the contract and publishes stubs for consumers.

## Stub, contract or end-to-end?

| | WIREMOCK STUB (P.30) | CONTRACT TEST | END-TO-END TEST (P.34) |
|---|---|---|---|
| **Catches** | your handling of errors, timeouts, odd data | the provider changing what consumers rely on | broken wiring across the whole system |
| **Misses** | the real service changing | behaviour beyond the request/response shape | rare failures — and it is slow to find the cause |
| **Runs** | consumer build, milliseconds | both builds, seconds | a deployed environment, minutes |

04 · INTEGRATION & API TESTING

# Time, randomness, threads & flaky tests

A test that passes and fails without any code change is **flaky**. Flaky tests are worse than none: people learn to press "re-run", and real failures get ignored. Almost every cause is something the test doesn't control.

TrickyThingsTest.java

```java
class TrickyThingsTest {

  @Test void timeComesFromAnInjectedClock() {
    Instant newYearsEve = Instant.parse("2026-12-31T23:59:59Z");
    Clock clock = Clock.fixed(newYearsEve, ZoneOffset.UTC);
    var service = new OrderService(new InMemoryOrderRepository(),
        (c, m) -> "PAY", (to, s, b) -> {}, new DiscountPolicy(), clock);

    Order o = service.placeOrder("a@example.com",
        List.of(new Item("PEN", Money.of("1"), 1)));

    assertThat(o.placedAt()).isEqualTo(newYearsEve);
  }

  @Test void randomnessComesFromASeed() {
    var a = new Random(42).ints(5, 0, 100).boxed().toList();
    var b = new Random(42).ints(5, 0, 100).boxed().toList();
    assertThat(a).isEqualTo(b);  // same seed, same sequence
  }

  @Test void filesGoInAFreshTempDir(@TempDir Path dir) throws Exception {
    Path report = dir.resolve("report.csv");
    Files.writeString(report, "sku,qty\nPEN,2\n");
    assertThat(Files.readAllLines(report))
        .containsExactly("sku,qty", "PEN,2");
  }  // JUnit deletes dir afterwards

  @Test void concurrentSavesAreAllKept() throws Exception {
    var repo = new InMemoryOrderRepository();
    int threads = 16, perThread = 500;
    var start = new CountDownLatch(1);
    try (ExecutorService pool = Executors.newFixedThreadPool(threads)) {
      for (int t = 0; t < threads; t++) {
        int base = t * perThread;
        pool.submit(() -> {
          start.await();  // release all threads at once
          for (int i = 0; i < perThread; i++)
            repo.save(new Order("o" + (base + i), "x@example.com",
                List.of(), Money.of("0"), Instant.EPOCH));
          return null;
        });
      }
      start.countDown();
    }  // close() waits for all tasks
    for (int i = 0; i < threads * perThread; i++)
      assertThat(repo.findById("o" + i)).isPresent();
  }

  @Test void slowCodeFailsFastInsteadOfHangingTheBuild() {
    assertTimeoutPreemptively(Duration.ofSeconds(1),
        () -> new DiscountPolicy().percentFor(Money.of("1")));
  }
}
```

real output — 5 / 5 passed

```
TrickyThingsTest ✔
├─ randomnessComesFromASeed() ✔
├─ timeComesFromAnInjectedClock() ✔
├─ filesGoInAFreshTempDir(Path) ✔
├─ concurrentSavesAreAllKept() ✔
└─ slowCodeFailsFastInsteadOfHangingTheBuild() ✔
```

## Where flakiness comes from

| CAUSE | FIX |
|---|---|
| **Real time** `LocalDate.now()` breaks at midnight, month end, DST | Inject a `Clock`; use `Clock.fixed` |
| **Randomness** UUIDs, shuffles, random test data | Seed it, or inject the generator |
| **Sleeping** `Thread.sleep(500)` is too short on a busy CI box | Wait for a condition (Awaitility) with a timeout |
| **Shared state** Static fields, a shared DB row, a singleton cache | Fresh fixtures per test; unique IDs; reset in `@AfterEach` |
| **Test order** Test B only passes after test A | Each test sets up its own data; run in random order to find these |
| **Time zone / locale** Passes in Bengaluru, fails on a UTC build agent | Set zone and locale explicitly in code and tests |
| **Network** Calls to real external services | Stub at the boundary; containers for owned dependencies |
| **Concurrency** Races appear one run in a hundred | Latches to line threads up; repeat; stress in a separate job |

- **Clock.fixed** makes "now" a constant; production passes `Clock.systemUTC()`.
- **new Random(42)** gives the same sequence on every run and every machine.
- **@TempDir** — no leftovers between runs, no clashes between parallel tests.
- **CountDownLatch** releases 16 threads together so they really overlap: 8,000 concurrent saves, none lost.
- **assertTimeoutPreemptively** stops a hung test instead of hanging the whole build.
- For async code, poll for a condition with **Awaitility** (`await().atMost(5, SECONDS).until(…)`) instead of sleeping.

**RULE OF THUMB**

Treat a flaky test like a bug. Quarantine it (a separate, non-blocking job), open a ticket, and fix or delete it within days — never add an automatic "retry until green" to the main build.

04 · INTEGRATION & API TESTING

# Concurrency bugs: jcstress

`count++` looks like one step; it is three — read, add, write. Two threads can both read 0 and both write 1. An ordinary test almost never sees it: the window is nanoseconds. **jcstress** (OpenJDK) runs a tiny test hundreds of millions of times, with the interpreter, C1 and C2 and compiler stress modes, and counts every outcome.

RacyCounterTest.java

```java
/** Two threads each do count++ once. Is the result always 2? */
@JCStressTest
@Outcome(id = "2", expect = ACCEPTABLE, desc = "both counted")
@Outcome(id = "1", expect = ACCEPTABLE_INTERESTING,
    desc = "lost update: count++ is read, add, write")
@State
public class RacyCounterTest {
  int count;

  @Actor
  public void first() {
    count++;
  }

  @Actor
  public void second() {
    count++;
  }

  @Arbiter
  public void result(I_Result r) {
    r.r1 = count;
  }
}
```

AtomicCounterTest.java

```java
/** The fix. A lost update would now fail the run. */
@JCStressTest
@Outcome(id = "2", expect = ACCEPTABLE, desc = "both counted")
@Outcome(id = "1", expect = FORBIDDEN, desc = "lost update")
@State
public class AtomicCounterTest {
  final AtomicInteger count = new AtomicInteger();

  @Actor
  public void first() {
    count.incrementAndGet();
  }

  @Actor
  public void second() {
    count.incrementAndGet();
  }

  @Arbiter
  public void result(I_Result r) {
    r.r1 = count.get();
  }
}
```

real output · jcstress 0.16 · mvn compile exec:exec (-m quick) · GitHub Actions, 4 CPUs · whole run 162 s

```
RacyCounterTest — results across all configurations
  RESULT      SAMPLES     FREQ       EXPECT  DESCRIPTION
       1  200,651,963   18.00%  Interesting  lost update: count++ is read, add, write
       2  914,222,425   82.00%   Acceptable  both counted

AtomicCounterTest — results across all configurations
  RESULT      SAMPLES     FREQ       EXPECT  DESCRIPTION
       1            0    0.00%   Forbidden  lost update
       2  608,342,548  100.00%  Acceptable  both counted
```

## Lost updates, by JIT mode

| ACTOR 1 | ACTOR 2 | FORKS | LOST |
|---|---|---|---|
| **Interpreter** | Interpreter | 2 | 0.99–11.67% |
| **Interpreter** | C1 | 2 | 0.24–2.80% |
| **Interpreter** | C2 | 4 | 0.14–2.84% |
| **C1** | Interpreter | 2 | 0.22–2.83% |
| **C1** | C1 | 2 | 10.67–67.08% |
| **C1** | C2 | 4 | 6.81–11.44% |
| **C2** | Interpreter | 4 | 0.19–3.80% |
| **C2** | C1 | 4 | 6.79–12.43% |
| **C2** | C2 | 4 | 4.94–60.70% |

- **200,651,963 lost updates** in 1,114,874,388 runs (18.00%). The rate depended on how each actor was compiled: from 0.14% to 67.08%.
- `ACCEPTABLE_INTERESTING` reports an outcome but passes; `FORBIDDEN` fails the run — so the atomic version guards the fix.
- Use it for code that shares memory without locks: custom counters and caches, lazy initialisation, publishing objects to other threads.
- Ordinary code: avoid shared mutable state, use `java.util.concurrent`.
- Run `-m quick` in CI; `default` or `tough` before a release. Tests live in `src/main`, by jcstress convention.

CHAPTER FIVE

# 05 Beyond correctness

Passing unit and integration tests prove the parts are right. This chapter checks the whole system from the outside: real browsers clicking real pages, real load on real servers, and an attacker's view of your code.

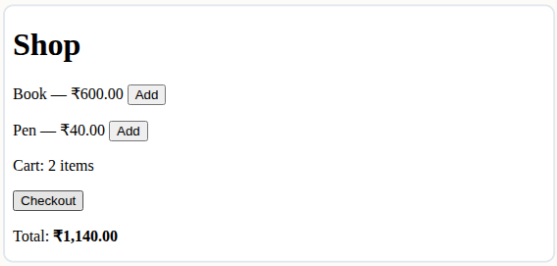
## End-to-end & UI testing

| TOOL | BROWSERS | NOTES | LATEST |
|---|---|---|---|
| **Playwright for Java** | Chromium, Firefox, WebKit | Auto-waits for elements; traces with screenshots; one API for browsers and HTTP | Microsoft · 1.63 |
| **Selenium WebDriver** | Every major browser | W3C WebDriver standard; Selenium Grid runs tests in parallel across machines | 4.51 |
| **Cypress** | Chromium-family, Firefox, WebKit (exp.) | Runs inside the browser; tests are JavaScript/TypeScript only — not Java | JS/TS |
| **REST Assured** | — (HTTP) | Java DSL for end-to-end API checks: `given().when().get().then()` | 6.1 |

CheckoutE2EIT — real Chromium, headless  2 ✔ · 10.68 s

```java
@Test
void twoBooksGetTheBulkDiscount() {
  var checkout = new CheckoutPage(page);
  checkout.addToCart("Book");
  checkout.addToCart("Book");
  checkout.checkout();

  assertThat(checkout.total()).hasText("₹1,140.00"); // waits
}
```




Playwright's own screenshot from the CI run, after checkout

### Choosing locators (Playwright's order)

1. **Role + name** · `getByRole(BUTTON, name "Checkout")` — what users and screen readers see
2. **Text** · `getByText("Order confirmed")` — user-visible, but changes with copy
3. **Label / placeholder** · `getByLabel("Email")` for form fields
4. **Test id** · `getByTestId` reads `data-testid` by default — stable, invisible to users
5. **CSS / XPath** · `div.cart > span:nth-child(3)` — last resort; breaks on any layout change

Real run on GitHub Actions: Playwright for Java 1.63.0 drove Chromium against a small shop page; both journeys passed, and each test saved a screenshot and a trace ( `target/*-trace.zip` , open with `playwright show-trace` ). The total appears 300 ms after checkout — the assertion waited for it, no sleep.

**Page Objects**
Wrap each page in a class with intent-revealing methods. When the HTML changes, you fix one class, not fifty tests.

**Never sleep**
Wait for a condition, not a duration. Playwright's locators and web-first assertions retry automatically until a timeout.

**Keep them few**
Cover the journeys that make money or lose data — sign-up, checkout, payment — plus a few read-only smoke tests in production after each deploy.

**Own your data**
Create users and products through the API at the start of each test; don't depend on what is already in the environment.

**Accessibility**
Run axe-core (Deque publishes Java bindings for Selenium and Playwright) on key pages; it flags missing labels and poor contrast automatically.

**Traces, not guesses**
Keep Playwright traces and screenshots as CI artefacts. Comparing screenshots also catches visual regressions — mask dates and other changing areas.

05 · BEYOND CORRECTNESS

# Performance testing

## Kinds of performance test

| | | |
|---|---|---|
| **Load** | Expected peak traffic, held steady | Does it meet the latency target at normal peak? |
| **Stress** | Ramp beyond the peak until it breaks | Where is the limit, and does it fail gracefully? |
| **Spike** | A sudden jump: 10× in seconds | Do autoscaling, queues and pools cope? |
| **Soak** | Normal load for hours | Memory leaks, connection leaks, growing queues |
| **Scalability** | Repeat with 1, 2, 4 instances | Does capacity grow with hardware? |

## Tools

| | | |
|---|---|---|
| **Gatling** | Java, Kotlin or Scala DSL; load tests as code; HTML reports | **3.16** |
| **Apache JMeter** | GUI to build test plans, CLI to run them; many protocols | **5.6.3** |
| **Grafana k6** | Tests in JavaScript; single binary; good CI fit | **2.3** |
| **JMH** | Microbenchmarks for one method; handles JIT warm-up and dead-code elimination | **1.37** |

RULE OF THUMB

Report **percentiles**, not averages. p99 = 99% of requests were at least this fast. A page that makes 100 calls has a $1 - 0.99^{100} \approx 63\%$ chance that at least one of them is slower than p99.

## A real load test — and what it found

LoadSmoke.java — virtual users against OrderApi

```
try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
  for (int u = 0; u < users; u++) {
    pool.submit(() -> {
      while (System.nanoTime() < end) {
        long t0 = System.nanoTime();
        try {
          var res = http.send(req, HttpResponse.BodyHandlers.ofString());
          if (res.statusCode() != 200) errors.inc();
        } catch (Exception e) { errors.inc(); }
        if (t0 > warmUpEnd) latencies.add(System.nanoTime() - t0);
      }
      return null;
    });
  }
}
```

real output · 50 users · 2 CPUs

```
users=50  requests=12,150  errors=0
throughput=1,215 req/s
p50     41.00 ms
p90     41.24 ms
p95     42.00 ms
p99     43.04 ms
p99.9   56.14 ms
max     58.44 ms
mean    41.15 ms
```

| RUN (10 S MEASURED, AFTER 3 S WARM-UP) | REQ/S | P50 MS | P95 MS | P99 MS | MAX MS | MEAN MS |
|---|---|---|---|---|---|---|
| **1 user** | 24 | 41.99 | 42.09 | 42.26 | 43.96 | 42.01 |
| **10 users** | 244 | 41.00 | 41.12 | 42.02 | 44.13 | 41.04 |
| **50 users** | 1,215 | 41.00 | 42.00 | 43.04 | 58.44 | 41.15 |
| **1 user · TCP_NODELAY** | 4,721 | 0.19 | 0.31 | 0.35 | 7.56 | 0.21 |
| **50 users · TCP_NODELAY** | 13,997 | 3.09 | 7.66 | 10.71 | 21.38 | 3.57 |

**The finding.** Every request took about 42 ms — even with a single user, so it wasn't load. Our handler sends the headers and the body as two separate small writes; with Nagle's algorithm on, the second waits for the client's delayed ACK — the Nagle/delayed-ACK stall (the companion book's chapter 1 measures it packet by packet). Starting the JVM with `-Dsun.net.httpserver.nodelay=true` (sets `TCP_NODELAY`) cut the single-user p50 from 42 ms to 0.19 ms. No unit test could have found this. The companion book, *Java Performance Deep Dives*, takes this stall apart packet by packet.

**Little's Law checks your numbers:** users in flight = throughput × time in system. 50 users ÷ 1,215 req/s = 41.2 ms (measured mean 41.1 ms), and 50 ÷ 13,997 = 3.6 ms (measured 3.6 ms). If your tool's numbers don't satisfy it, the load generator itself is the bottleneck. These figures come from this guide's 2-CPU sandbox: compare runs, not absolute numbers.

WATCH OUT

**Coordinated omission:** a closed-loop tool like this one stops sending while the server stalls, so slow periods produce fewer samples and percentiles look better than reality. Open-model tools (Gatling's `constantUsersPerSec`, k6 arrival-rate executors) keep sending at a fixed rate.

05 · BEYOND CORRECTNESS

# Security & resilience testing

Security testing is mostly automation you add to the pipeline once, plus security cases you write like any other test. The scanners find known problems; your own tests prove your rules — who may see what — actually hold.

| KIND | WHAT IT DOES | JAVA-FRIENDLY TOOLS | WHEN |
|---|---|---|---|
| **SAST** | Reads source or bytecode for dangerous patterns | SpotBugs + Find Security Bugs, Semgrep, CodeQL | Every build |
| **SCA** | Finds libraries with known vulnerabilities (CVEs) | OWASP Dependency-Check, Dependabot, Renovate | Every build + daily |
| **Secrets** | Finds keys and passwords committed to git | gitleaks, GitHub secret scanning | Every push |
| **DAST** | Attacks the running app from outside | ZAP (now "ZAP by Checkmarx"), Burp Suite | Nightly, on staging |
| **Fuzzing** | Feeds generated, malformed input to find crashes | Jazzer (coverage-guided, JUnit integration) | Nightly / continuous |
| **Pen test** | Humans think like attackers | Internal or external security team | Before big releases |

## Security tests you write yourself

OrderApiTest.java — hostile inputs ✔ passes

```
@Test void pathTricksAreJustNotFound() throws Exception {
  for (String path : List.of("/orders/..%2f..%2fetc%2fpasswd",
                             "/orders/ord-1'%20OR%20'1'='1",
                             "/orders/" + "x".repeat(5_000))) {
    var res = send("GET", path);
    assertThat(res.statusCode()).isEqualTo(404);
    assertThat(res.body())              // no stack traces, no internals
        .isEqualTo("{\"error\":\"order not found\"}");
  }
}
```

Path traversal, SQL-looking IDs and a 5,000-character ID all get the same plain 404 — no stack trace, no SQL error, no file contents. This is a normal JUnit test; it runs on every build with the rest of the suite. The cards below are more ideas to turn into tests like this one.

**Access control**
User A asks for `/orders/{id}` of user B → expect 403 or 404, never 200.

**Injection**
Names like `' OR 1=1 --` and `<script>` are stored and shown as plain text.

**Authentication**
Expired, tampered and missing tokens all get 401; lock-out after repeated failures.

**Input limits**
Huge bodies, deep JSON, 10,000-item lists: rejected quickly, not crashing the server.

**Error leakage**
Errors return a generic message — no stack traces, SQL or internal hostnames.

**Headers**
Responses with personal data send `Cache-Control: no-store`; HTTPS-only cookies.

WATCH OUT

**Broken access control** is A01, the first entry, in the OWASP Top 10:2025 (and was in 2021) — for example one user reading another's data by changing an ID. A scanner does not know which user may see which record; write explicit tests for it on every endpoint that takes an ID.

RULE OF THUMB

Fail the build on **new** high-severity findings, and track the existing backlog separately, so that the first scan of an old code base does not block every build.

## Resilience: test the failure, not just the success

**Timeouts**
Point the client at a server that never answers; the call must give up in time.

**Slow dependency**
Add latency with Toxiproxy (a Testcontainers module); check pools don't exhaust.

**Partial failure**
Kill one instance or the cache during a load test; errors should stay bounded.

**Retries**
Make the dependency fail twice then succeed; assert exactly three calls and idempotent effects.

Chaos engineering takes the same idea to production-like environments: inject failures deliberately (kill pods, add latency) and check the system degrades the way you designed it to.

05 · BEYOND CORRECTNESS

# Testing in production: flags and synthetic checks

Tests before release can't see the real network, data and load. Two habits make what happens after a deploy safer: every **feature flag** is tested both ways before it ships, and a **synthetic check** — the same code in a test and after every deploy — keeps asking "does it still work?".

CheckoutPricingTest — both sides of the flag — 3 ✔ · 0.116 s

```java
/** Every flag is two code paths: test both, every build. */
class CheckoutPricingTest {
  @ParameterizedTest(name = "flag={0}, {1} items → {2}")
  @CsvSource({
      "false, 3, 100000",
      "true,  3,  90000",
      "true,  2, 100000"})
  void bulkDiscountFollowsTheFlag(boolean on, int items,
      long total) {
    var pricing = new CheckoutPricing(new Features(on));
    assertThat(pricing.total(100_000, items)).isEqualTo(total);
  }
}
```

FeatureWiringTest — the property reaches the bean — 2 ✔ · 0.609 s

```java
/** The property really reaches the bean; no web server, ~ms. */
class FeatureWiringTest {
  ApplicationContextRunner ctx = new ApplicationContextRunner()
      .withUserConfiguration(FeatureConfig.class);

  @Test
  void offWhenThePropertyIsMissing() {
    ctx.run(c -> assertThat(c.getBean(CheckoutPricing.class)
        .total(100_000, 3)).isEqualTo(100_000));
  }

  @Test
  void onWithThePropertySet() {
    ctx.withPropertyValues("shop.features.bulk-discount=true")
        .run(c -> assertThat(c.getBean(CheckoutPricing.class)
            .total(100_000, 3)).isEqualTo(90_000));
  }
}
```

SyntheticCheck.java — what a user does

```java
public static List<String> run(URI base) {
  List<String> failures = new ArrayList<>();
  check(failures, base, "/actuator/health", "\"UP\"");
  check(failures, base, "/prices/book-1", "\"paise\"");
  return failures;
}

private static void check(List<String> failures, URI base,
    String path, String mustContain) {
  long t0 = System.nanoTime();
  String result;
  try {
    var req = HttpRequest.newBuilder(base.resolve(path))
        .timeout(Duration.ofSeconds(5)).build();
    var res = HTTP.send(req,
        HttpResponse.BodyHandlers.ofString());
    boolean ok = res.statusCode() == 200
        && res.body().contains(mustContain);
    result = (ok ? "✔ " : "✘ ") + path
        + "  " + res.statusCode();
  } catch (Exception e) {
    result = "✘ " + path + "  "
        + e.getClass().getSimpleName();
  }
  long ms = (System.nanoTime() - t0) / 1_000_000;
  System.out.println(result + "  " + ms + " ms");
  if (result.startsWith("✘")) {
    failures.add(result);
  }
}
```

SyntheticCheckIT — before it ships — 1 ✔ · 9.730 s

```java
/** The production check, run before the app ships. */
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@Import(RedisContainer.class)
class SyntheticCheckIT {
  @Value("${local.server.port}") int port;

  @Test
  void passesAgainstAHealthyApp() {
    var base = URI.create("http://localhost:" + port);
    assertThat(SyntheticCheck.run(base)).isEmpty();
  }
}
```

real output · CI · against the running jar

```
$ java -cp target/classes shop.synthetic.SyntheticCheck
http://localhost:8080
✔ /actuator/health  200  88 ms
✔ /prices/book-1  200  55 ms
synthetic check passed
exit code 0
```

real output · same app, Redis stopped

```
$ docker stop redis; java -cp … SyntheticCheck
http://localhost:8080
✘ /actuator/health  503  2096 ms
✘ /prices/book-1  500  2039 ms
synthetic check FAILED: 2
exit code 1
```

- **Every flag is two code paths.** Test both on every build; a flag that has been on everywhere for a release is deleted, with its tests.
- **One synthetic check, three places:** the test above before the app ships, CI against the packaged jar, and production on a schedule after each deploy.
- With Redis down the health endpoint said 503 and the check failed with exit code 1 — that is what pages someone.

GOOD TO KNOW

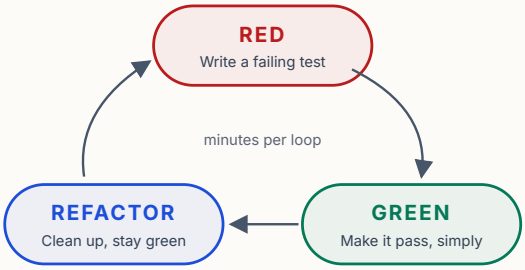
**Not run here — practice:** ship behind a flag that is off; turn it on for a **canary** (a small share of traffic); compare its error rate and latency with the old version; roll back automatically if they are worse; then turn it on for everyone.

CHAPTER SIX

# 06 Quality & process

Tests are only as useful as the habits around them: when you write them, how you know they are good enough, and how they run on every change. This chapter covers TDD, coverage, mutation testing, CI — and questions to check your understanding of all of it.

## Test-driven development




Kent Beck's *Test-Driven Development: By Example* (2002) describes a short loop: write a test for behaviour that doesn't exist yet, watch it fail, write just enough code to pass, then improve the design while the tests stay green.

- **Red first** proves the test can fail — a test that never failed may not test anything.
- **Simplest code** that passes, even if it looks naive; the next test forces the real logic.
- **Refactor** both code and tests; the tests are your safety net.

## A real TDD loop: `PasswordPolicy`

step 1 · tests written, class just compiles

```
public class PasswordPolicy {
  public List<String> problems(String password) {
    return List.of();  // step 1: just enough to compile
  }
}
```

real output · RED — 3 failed

```
JUnit Jupiter
  '-- PasswordPolicyTest [OK]
    +-- passwordWithoutUpperCaseIsReported() [X]
    +-- shortPasswordIsReported() [X]
    +-- strongPasswordHasNoProblems() [OK]
    '-- passwordWithoutDigitIsReported() [X]

shortPasswordIsReported():
  Expecting actual:
    []
  to contain exactly (and in same order):
    ["at least 12 characters"]
  but could not find the following elements:
    ["at least 12 characters"]
```

step 2 · just enough code

```
public class PasswordPolicy {
  public List<String> problems(String password) {
    List<String> problems = new ArrayList<>();
    if (password.length() < 12)
      problems.add("at least 12 characters");
    if (password.chars().noneMatch(Character::isDigit))
      problems.add("a digit");
    if (password.chars().noneMatch(Character::isUpperCase))
      problems.add("an upper-case letter");
    return problems;
  }
}
```

real output · GREEN — 4 / 4 passed

```
PasswordPolicyTest ✔
├─ passwordWithoutUpperCaseIsReported() ✔
├─ shortPasswordIsReported() ✔
├─ strongPasswordHasNoProblems() ✔
└─ passwordWithoutDigitIsReported() ✔
```

**Step 3 · refactor:** with all four green, we could pull the rules into a list of `(predicate, message)` pairs — and re-run to prove nothing changed.

Honest note: we wrote all four tests before the code, so three failed at once. Strict TDD adds one test at a time. The strong-password test passed even in the red step, because an empty list happens to be its right answer — a reminder to watch *each* new test fail before trusting it.

## BDD: tests in the customer's words

orders.feature · Gherkin example

```
Feature: Order discounts
  Scenario: Orders of ₹1000 or more get 5% off
    Given a cart with 2 books at ₹600 each
    When the customer places the order
    Then the order total is ₹1140.00
    And a confirmation e-mail is sent
```

Behaviour-driven development writes acceptance tests as **Given / When / Then** scenarios that product owners can read. Cucumber-JVM maps each line to a Java step method. Worth it when non-developers really read and write the scenarios; otherwise plain JUnit tests with good names are cheaper.

06 · QUALITY & PROCESS

# Coverage & mutation testing

## Code coverage (JaCoCo)

| | | |
|---|---|---|
| **Instructions** | C0 | Java bytecode instructions executed — the finest grain |
| **Branches** | C1 | Both outcomes of every `if` and `switch` case |
| **Lines** | | A line counts as covered when at least one of its instructions ran |
| **Complexity** | | Cyclomatic complexity: paths that would need tests |
| **Methods / classes** | | Executed at least once |

Coverage tells you what the tests **ran**, not what they **checked**. A test with no assertions can reach 100%. Use it to find untested code, not as a score to chase.

## Mutation testing (PIT)

A tool changes your code in small ways — a **mutant** — and re-runs the tests. If they still pass, the mutant **survived**: your tests missed a bug. Mutation score = killed ÷ total. The idea goes back to Lipton (1971) and DeMillo, Lipton & Sayward (1978).

| | |
|---|---|
| CONDITIONALS_BOUNDARY | `<` ↔ `<=`, `>=` ↔ `>` |
| NEGATE_CONDITIONALS | `==` ↔ `!=`, `<` ↔ `>=` |
| MATH | `+` ↔ `-`, `*` ↔ `/` |
| INCREMENTS | `i++` ↔ `i--` |
| INVERT_NEGS | `-x` → `x` |
| VOID_METHOD_CALLS | removes a call to a void method |
| …_RETURNS | empty, null, true/false and 0 returns |

## A mutant, by hand: does the test suite notice?

WeakDiscountPolicyTest.java

```java
/** 100% line + branch coverage, yet the bug survives. */
class WeakDiscountPolicyTest {
  DiscountPolicy policy = new DiscountPolicy();

  @Test void small() {
    assertEquals(0, policy.percentFor(Money.of("500")));
  }
  @Test void medium() {
    assertEquals(5, policy.percentFor(Money.of("2000")));
  }
  @Test void large() {
    assertEquals(10, policy.percentFor(Money.of("6000")));
  }
}
```

the mutant · CONDITIONALS_BOUNDARY

```
-    if (subtotal.compareTo(Money.of("1000")) >= 0) return 5;
+    if (subtotal.compareTo(Money.of("1000")) > 0) return 5;
```

weak tests vs MUTANT · real output — mutant survived

```
JUnit Jupiter
  '-- WeakDiscountPolicyTest [OK]
    +-- medium() [OK]
    +-- large() [OK]
    '-- small() [OK]
```

boundary tests (p.06) vs MUTANT — mutant killed

```
JUnit Jupiter
  '-- DiscountPolicyTest [OK]
    '-- percentForSubtotal(String, int) [OK]
      +-- ₹0 → 0% off [OK]
      +-- ₹999.99 → 0% off [OK]
      +-- ₹1000 → 5% off [X] expected: <5> but was: <0>
      +-- ₹1000.01 → 5% off [OK]
      +-- ₹4999.99 → 5% off [OK]
      +-- ₹5000 → 10% off [OK]
      '-- ₹250000 → 10% off [OK]
```

The weak tests execute every line and both outcomes of each `if`, so line and branch coverage are both 100% — yet they let the bug through. The boundary test at exactly ₹1000 kills it. PIT does this automatically for thousands of mutants: `mvn test-compile org.pitest:pitest-maven:mutationCoverage` (pitest-maven 1.30.0, with the `pitest-junit5-plugin` for JUnit 5/6 tests).

## Test smells to refactor away

| SMELL | LOOKS LIKE | FIX |
|---|---|---|
| **Assertion-free test** | Calls code, checks nothing — still counts as coverage | Assert the outcome, or delete the test |
| **Eager test** | One test checks five behaviours; the name says “testOrder” | Split it: one behaviour, one name |
| **Mystery guest** | Expected values come from a file or DB row you can’t see | Build the data inside the test (builders) |
| **Fragile test** | Breaks on harmless refactors; verifies every internal call | Assert results, not implementation steps |
| **Sleepy test** | `Thread.sleep` to wait for async work | Wait for a condition with a timeout |

**RULE OF THUMB**

Run mutation testing where it matters — pricing, permissions, money — or only on files changed in a pull request; it re-runs tests for every mutant, so it is slow.

06 · QUALITY & PROCESS

# Mutation testing with PIT: a worked run

We ran **PIT 1.30.0** with **pitest-junit5-plugin 1.2.3** on the shop example: every production class mutated with the DEFAULTS group, every unit and integration test run against each mutant. Below is what it found and how we fixed it.

## Run 1 — what PIT reported

real PIT summary · run 1 — 39/40 detected

```
Line Coverage (for mutated classes only): 76/77 (99%)
22 tests examined
Generated 40 mutations Killed 39 (98%)
Mutations with no coverage 0. Test strength 98%
```

"22 tests examined" counts test *classes*; PIT ran each mutant against only the tests that cover its line. **Run 2**, after the fix: 40/40 detected — **100%**. With the larger STRONGER group: 50/50 (100%).

**GOOD TO KNOW**

Before writing a test for a survivor, ask whether the behaviour matters. Some mutants are **equivalent** — the change can't alter any observable result — and no test can kill them. Accept those rather than writing a meaningless test.

## The survivor

**OrderApi.stop()**, line 24: PIT deleted the call `server.stop(0)` and **all 43 tests still passed**. Nothing checked that stopping the API actually frees its port — a resource leak would ship unnoticed.

the test we added to kill it — ✔ passes

```
/** Kills the mutant PIT found: deleting server.stop(0). */
class OrderApiLifecycleTest {

  @Test void stopReleasesThePort() throws Exception {
    var api = new OrderApi(new InMemoryOrderRepository());
    int port = api.start();

    api.stop();

    assertThatThrownBy(() -> new Socket("127.0.0.1", port).close())
        .isInstanceOf(ConnectException.class);  // port is free
  }
}
```

## Results by mutator

| MUTATOR | MADE | KILLED | TIMED OUT | SURVIVED · RUN 1 | SURVIVED · RUN 2 |
|---|---|---|---|---|---|
| VOID_METHOD_CALLS | 10 | 5 | 4 | 1 | 0 |
| NEGATE_CONDITIONALS | 10 | 10 | 0 | 0 | 0 |
| NULL_RETURNS | 7 | 7 | 0 | 0 | 0 |
| CONDITIONALS_BOUNDARY | 5 | 5 | 0 | 0 | 0 |
| PRIMITIVE_RETURNS | 4 | 4 | 0 | 0 | 0 |
| EMPTY_RETURNS | 3 | 3 | 0 | 0 | 0 |
| MATH | 1 | 1 | 0 | 0 | 0 |

## What PIT actually changed — a sample of the 40 mutants

| CLASS · LINE | PIT'S DESCRIPTION | TESTS RUN | RESULT |
|---|---|---|---|
| **DiscountPolicy · 7** | changed conditional boundary | 12 | KILLED |
| **Money · 19** | Replaced integer subtraction with addition | 8 | KILLED |
| **OrderService · 32** | removed call to EmailSender::send | 6 | KILLED |
| **PasswordPolicy · 16** | replaced return value with Collections.emptyList | 4 | KILLED |
| **InMemoryOrderRepository · 15** | replaced return value with Optional.empty | 5 | KILLED |
| **OrderApi · 20** | removed call to HttpServer::start | 0 | TIMED_OUT |
| **OrderApi · 24** | removed call to HttpServer::stop | 2 | KILLED (run 2) |

Note what is *not* there: `payments.charge(…)` returns a value, so the DEFAULTS group never removes it — only void calls are removed. Larger groups (STRONGER, ALL) add mutators for those cases.

06 · QUALITY & PROCESS

# PIT in your build and CI

pom.xml — the same settings as our run, as Maven config

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.30.0</version>
  <dependencies>
    <dependency>  <!— runs JUnit 5/6 tests —>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>1.2.3</version>
    </dependency>
  </dependencies>
  <configuration>
    <targetClasses><param>shop.*</param></targetClasses>
    <targetTests><param>shop.*Test</param></targetTests>
    <mutators><mutator>DEFAULTS</mutator></mutators>
    <mutationThreshold>90</mutationThreshold>  <!— fail below 90% —>
    <coverageThreshold>85</coverageThreshold>
    <withHistory>true</withHistory>           <!— incremental —>
    <threads>4</threads>
    <outputFormats><param>HTML</param><param>XML</param></outputFormats>
  </configuration>
</plugin>
```

## Run it

| COMMAND | DOES |
|---|---|
| `mvn test-compile pitest:mutationCoverage` | full analysis; HTML report in `target/pit-reports` |
| `-DtargetClasses=shop.pricing.*` | narrow the run to one package |
| `-DwithHistory` | reuse the last run's results (incremental) |
| `-Dmutators=STRONGER` | more mutators; slower, finds more gaps |

## Making it fast enough for CI

- **Incremental:** with history on, only changed code and tests are re-analysed.
- **Pull requests:** PIT 1.30 has no git-diff goal. Narrow `targetClasses` to the changed module (or use Arcmutate's commercial add-ons); run everything nightly.
- **Target it:** modules with money, permissions and business rules first.
- Our run: 31 s for 40 mutants on 2 CPUs, because PIT re-runs only the tests covering each mutant.

## Reading a mutant's status

| STATUS | MEANING | DETECTED? |
|---|---|---|
| **KILLED** | a test failed — the bug was caught | **yes** |
| **TIMED_OUT** | mutant made the code hang (e.g. a removed `start()`); counted as detected | **yes** |
| **SURVIVED** | every covering test still passed — a gap in your assertions | no |
| **NO_COVERAGE** | no test even runs this line | no |
| **NON_VIABLE** | the mutated bytecode was invalid | — |
| **MEMORY_ERROR / RUN_ERROR** | the test JVM crashed | **yes** |

## Unit or integration tests?

PIT runs whatever tests you point it at. Ours included the HTTP integration tests, which is how it could detect mutants in `OrderApi` at all — no unit test touches that class. The cost: mutants that broke `start()` or the handler made tests hang until PIT's timeout (the 4 **TIMED_OUT** results).

**A practical split:** run PIT on every pull request with unit tests only (fast, precise), and nightly with integration tests included for the modules where wiring matters. Keep those tests quick — random ports, fakes, no sleeps.

## A workflow for survivors

**1 · Read it**
Open the HTML report: the line, the change PIT made, and which tests covered it.

**2 · Does it matter?**
Would a user, caller or log notice the change? If not, it may be equivalent or the code may be dead.

**3 · Test or delete**
Write the assertion that fails on the mutant — or delete code that nothing depends on.

**4 · Gate the trend**
Fail the build when the score drops below the threshold, so it can't quietly erode.

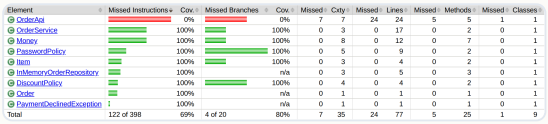
06 · QUALITY & PROCESS

# Coverage for unit *and* integration tests

By default most builds measure coverage for unit tests only, so code exercised only by integration tests looks untested. We ran the **JaCoCo 0.8.15** agent twice — once for the 39 unit tests, once for the 5 API integration tests — and merged the results (real numbers from `coverage.sh`; classes without branches left out).

| CLASS | UNIT LINES | IT LINES | MERGED LINES | UNIT BRANCHES | IT BRANCHES | MERGED BRANCHES |
|---|---|---|---|---|---|---|
| **OrderService** | 100% | 0% | 100% | 100% | 0% | 100% |
| **DiscountPolicy** | 100% | 0% | 100% | 100% | 0% | 100% |
| **Money** | 100% | 42% | 100% | 100% | 50% | 100% |
| **PasswordPolicy** | 100% | 0% | 100% | 100% | 0% | 100% |
| **Item** | 100% | 75% | 100% | 100% | 50% | 100% |
| **OrderApi** | 0% | 100% | 100% | 0% | 100% | 100% |
| **TOTAL** | **69%** | **49%** | **100%** | **80%** | **30%** | **100%** |

- **Unit tests alone: 69% of lines** — `OrderApi` looks completely untested.
- **Integration tests alone: 49%** — they never touch the pricing or password rules.
- **Merged: 100% of lines and branches.** Each level covers what the other can't.
- Coverage is a floor, not a goal: PIT (p.40) is what showed the tests also *check* what they run.



| Element | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods | Missed | Classes |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| OrderApi | | 0% | | 0% | 7 | 7 | 24 | 24 | 5 | 5 | 1 | 1 |
| OrderService | | 100% | | 100% | 0 | 3 | 0 | 17 | 0 | 2 | 0 | 1 |
| Money | | 100% | | 100% | 0 | 8 | 0 | 12 | 0 | 7 | 0 | 1 |
| PasswordPolicy | | 100% | | 100% | 0 | 5 | 0 | 9 | 0 | 2 | 0 | 1 |
| Item | | 100% | | 100% | 0 | 3 | 0 | 4 | 0 | 2 | 0 | 1 |
| InMemoryOrderRepository | | 100% | | n/a | 0 | 3 | 0 | 5 | 0 | 3 | 0 | 1 |
| DiscountPolicy | | 100% | | 100% | 0 | 4 | 0 | 4 | 0 | 2 | 0 | 1 |
| Order | | 100% | | n/a | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| PaymentDeclinedException | | 100% | | n/a | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| Total | 122 of 398 | 69% | 4 of 20 | 80% | 7 | 35 | 24 | 77 | 5 | 25 | 1 | 9 |

JaCoCo's own HTML report, unit tests only



```
14.  public OrderApi(OrderRepository repository) { this.repository = repository; }
15.
16.  public int start() throws IOException {
17.    // port 0 = let the OS pick any free port
18.    server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
19.    server.createContext("/orders/", this::handle);
20.    server.start();
21.    return server.getAddress().getPort();
22.  }
23.
24.  public void stop() { server.stop(0); }
25.
26.  private void handle(HttpExchange ex) throws IOException {
27.    if (!ex.getRequestMethod().equals("GET")) {
28.      ex.getResponseHeaders().set("Allow", "GET");
29.      send(ex, 405, "{\"error\":\"method not allowed\"}");
30.      return;
31.    }
```

Red lines: never executed by unit tests — only the HTTP integration tests reach them

pom.xml — unit + IT coverage, merged, with a gate

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.15</version>
  <configuration>  <!-- report and check read the merged file -->
    <dataFile>${project.build.directory}/jacoco-merged.exec</dataFile>
  </configuration>
  <executions>
    <execution><id>unit</id>      <!-- for Surefire -->
      <goals><goal>prepare-agent</goal></goals></execution>
    <execution><id>it</id>        <!-- for Failsafe -->
      <goals><goal>prepare-agent-integration</goal></goals></execution>
    <execution><id>merge</id><phase>verify</phase>
      <goals><goal>merge</goal></goals>
      <configuration>
        <fileSets><fileSet>
          <directory>${project.build.directory}</directory>
          <includes><include>jacoco.exec</include>
                    <include>jacoco-it.exec</include></includes>
        </fileSet></fileSets>
        <destFile>${project.build.directory}/jacoco-merged.exec</destFile>
      </configuration></execution>
    <execution><id>report</id><phase>verify</phase>
      <goals><goal>report</goal></goals></execution>
    <execution><id>gate</id><phase>verify</phase>
      <goals><goal>check</goal></goals>
      <configuration>
        <rules><rule><element>BUNDLE</element><limits><limit>
          <counter>BRANCH</counter><value>COVEREDRATIO</value>
          <minimum>0.80</minimum>
        </limit></limits></rule></rules>
      </configuration></execution>
  </executions>
</plugin>
```

## How the pieces fit

- **prepare-agent** sets `argLine` for Surefire; **prepare-agent-integration** does the same for Failsafe, writing `jacoco-it.exec`.
- **merge** combines the `.exec` files; **report** turns them into HTML, XML and CSV.
- **check** fails the build when a rule isn't met — here 80% of branches.
- Multi-module builds: **report-aggregate** in a separate reporting module.
- Other agents in `argLine` (like the Mockito agent on p.16) must keep JaCoCo's value: write `@{argLine} -javaagent:…`.

**RULE OF THUMB**

Gate coverage on **new or changed code** (SonarQube's "new code" quality gate, or a diff-coverage tool) rather than the whole code base. Old, untested code then can't block every merge, but nothing new arrives untested.

06 · QUALITY & PROCESS

# Finding duplicate & redundant tests

Two tests duplicate each other when they **catch the same bugs**. Run PIT with `fullMutationMatrix=true` — it records every test that kills each mutant — then compare the tests' kill sets. The comparison is a small Java class with no dependencies; Maven or Gradle runs it right after PIT.

src/tools/shop/RedundantTests.java — comparing kill sets

```
List<String> tests = new ArrayList<>(kills.keySet());
tests.sort(Comparator.comparing((String t) -> -kills.get(t).size())
    .thenComparing(Comparator.naturalOrder()));
Map<Set<Integer>, List<String>> groups = new LinkedHashMap<>();
for (String t : tests)
  groups.computeIfAbsent(kills.get(t), k -> new ArrayList<>()).add(t);
for (List<String> g : groups.values())
  if (g.size() > 1)
    System.out.println("  " + String.join("  ≡  ", g) + "   ("
        + kills.get(g.get(0)).size() + " mutants)");

for (String a : tests)
  for (String b : tests)
    if (!a.equals(b) && kills.get(b).containsAll(kills.get(a))
        && kills.get(b).size() > kills.get(a).size()) {
      System.out.printf("  %s (%d) ⊂ %s (%d)%n", a,
          kills.get(a).size(), b, kills.get(b).size());
      break;
    }
```

real output · identical from Maven and Gradle

```
39 tests kill at least one mutant

Same mutants killed:
  DiscountPolicyTest.percentForSubtotal#4  ≡
DiscountPolicyTest.percentForSubtotal#5   (6 mutants)
  ItemTest.ValidQuantity.totalIsPriceTimesQty#2  ≡
ItemTest.ValidQuantity.totalIsPriceTimesQty#3   (5
mutants)
  PasswordPolicyTest.passwordWithoutDigitIsReported  ≡
PasswordPolicyTest.passwordWithoutUpperCaseIsReported
(5 mutants)
  OrderApiTest.pathTricksAreJustNotFound  ≡
OrderApiTest.unknownOrderReturns404   (4 mutants)
  ItemTest.InvalidQuantity.rejected#1  ≡
ItemTest.InvalidQuantity.rejected#2  ≡
ItemTest.InvalidQuantity.rejected#3  ≡  OrderApiTest
(2 mutants)
  MockitoFeaturesTest.consecutiveAnswersForRetries  ≡
MockitoFeaturesTest.finalClassesCanBeMockedByDefault
≡  MoneyTest.rejectsNegativeAmounts   (1 mutants)

Strict subset of another test: 29 tests
```

pom.xml · mvn test-compile pitest:mutationCoverage exec:java

```
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <version>3.6.4</version>
  <configuration>
    <mainClass>shop.RedundantTests</mainClass>
    <classpathScope>test</classpathScope>
    <arguments>
      <argument>target/pit-reports/mutations.xml</argument>
    </arguments>
  </configuration>
</plugin>
```

build.gradle.kts · gradle pitest findRedundantTests

```
tasks.register<JavaExec>("findRedundantTests") {
  group = "verification"
  classpath = sourceSets["tools"].runtimeClasspath
  mainClass = "shop.RedundantTests"
  args("build/reports/pitest/mutations.xml")   // from the pitest
task
  jvmArgs("-Dstdout.encoding=UTF-8")
}
```

**WATCH OUT**

Both builds ran on GitHub Actions (Apache Maven 3.10.0, Gradle 9.8.1, JDK 21.0.12.1): PIT killed 40 of 40 mutants and the checker printed the same report in both. Equal kill sets mean the tests *currently* catch the same bugs — not that they are the same test. Delete only when the inputs are in the same partition *and* the assertions are the same.

## A human verdict on each match

| MATCH | WHAT THE TESTS ARE | VERDICT |
|---|---|---|
| **ItemTest…rejected #1 ≡ #2 ≡ #3** | quantities 0, -1, MIN_VALUE | **Redundant** — same partition. Keep 0 (the boundary) and one negative. |
| **DiscountPolicyTest #4 ≡ #5** | ₹4999.99 and ₹5000 | **Keep both** — the two sides of a boundary. |
| **PasswordPolicyTest digit ≡ upper-case** | two different rules | **Keep both** — they specify different rules. |
| **OrderApiTest pathTricks ≡ unknownOrder** | both expect 404 | **Merge candidate** — the hostile-input loop already covers the plain 404. |
| **Unit test ⊂ OrderServiceTest** | e.g. `MoneyTest` | **Keep** — a failing broad test doesn't say *which* rule broke. |

06 · QUALITY & PROCESS

# Build plugins that guard test quality

Everything in this chapter can run inside the build, so quality is checked on every change rather than when someone remembers. Versions are the latest releases at the time of writing.

| CONCERN | MAVEN PLUGIN | GRADLE PLUGIN | NOTES |
|---|---|---|---|
| **Unit tests** | maven-surefire-plugin **3.6.0** | built-in `test` task | `Test*`, `*Test`, `*Tests`, `*TestCase` |
| **Integration tests** | maven-failsafe-plugin **3.6.0** | a second `Test` task | `IT*`, `*IT`, `*ITCase`; fails in `verify` |
| **Extra test source dirs** | build-helper-maven-plugin **3.6.2** | `sourceSets` (built-in) | e.g. `src/it/java` |
| **Coverage** | jacoco-maven-plugin **0.8.15** | `jacoco` (built-in) | gate: `check` / `jacocoTestCoverageVerification` |
| **Mutation testing** | pitest-maven **1.30.0** | `info.solidsoft.pitest` **1.19.0** | gate: `mutationThreshold` |
| **Copy-paste in tests** | maven-pmd-plugin **3.28.0** · `cpd-check` | `de.aaschmid.cpd` **3.5** · `cpdCheck` | both use PMD's CPD engine |
| **Tests that assert nothing** | maven-pmd-plugin · `check` | `pmd` (built-in) | rule `UnitTestShouldIncludeAssert` |
| **Bug patterns** | spotbugs-maven-plugin **4.10.5.0** | `com.github.spotbugs` **6.5.13** | null dereferences, bad `equals`, races |
| **Code style** | maven-checkstyle-plugin **3.6.0** | `checkstyle` (built-in) | keeps test code readable too |
| **Vulnerable dependencies** | dependency-check-maven **13.0.0** | `org.owasp.dependencycheck` **13.0.0** | OWASP Dependency-Check (SCA) |
| **Flaky-test retries** | Surefire `rerunFailingTestsCount` | `org.gradle.test-retry` **1.6.6** | reports a pass-on-retry as flaky |
| **Quality gate & history** | sonar-maven-plugin **5.8.0.7211** | `org.sonarqube` **7.5.0.8588** | coverage and duplication on new code |

## Copy-paste detection in test code — a real run

pom.xml · mvn generate-test-sources pmd:cpd-check

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.28.0</version>
  <configuration>
    <includeTests>true</includeTests>
    <minimumTokens>25</minimumTokens>
    <printFailingErrors>true</printFailingErrors>
  </configuration>
</plugin>
```

build.gradle.kts · gradle cpdCheck

```
cpd {
  toolVersion = "7.17.0"
  minimumTokenCount = 25
}
tasks.cpdCheck {
  reports { text.required = true }
}
```

real output · PMD 7.17.0 · tests that assert nothing

```
$ pmd check -d src/test/java -d src/it/java -R
…/UnitTestShouldIncludeAssert
(no violations, exit code 0)

$ pmd check -d demo/pmd -R …/UnitTestShouldIncludeAssert
demo/pmd/NoAssertTest.java:4:   UnitTestShouldIncludeAssert:
This unit test should include assert() or fail()
exit code 4
```

real output · 8 duplications (Maven: 8)

```
Found a 11 line (49 tokens) duplication in the following files:
Starting at line 27 of src/it/java/shop/SharedApi.java
Starting at line 34 of src/it/java/shop/SharedPostgres.java
    log("API started on " + BASE_URL);
  }                          // no stop(): JVM exit frees it
  static void log(String msg) {
    System.out.printf("[fork %s pid %d] %s%n",
        FORK, ProcessHandle.current().pid(), msg);
  }
  /** Unique per fork and test: forks never share rows. */
  static String uniqueId(String test) {
    return "f" + FORK + "-" + test;
```

- **The defaults miss test duplication.** Maven's CPD skips tests unless `includeTests` is on, and needs 100 tokens; the Gradle plugin checks every source set at 50. On this project: **0** duplications at 100 tokens, **0** at 50, **8** at 25.
- The finding above is real: our two singleton base classes (p.47) repeat `log()` and `uniqueId()` — a shared helper fixes it.
- Run alone, `pmd:cpd-check` skips the lifecycle, so folders added by build-helper are missed — our first run found 2, not 8. Run it after `generate-test-sources` or bind it to `verify`.
- Both fail the build on duplicates. PMD's `UnitTestShouldIncludeAssert` rule (left) finds tests that check nothing.
- The Gradle CPD plugin (3.5, Nov 2024) prints Gradle 9 deprecation warnings.

06 · QUALITY & PROCESS

# Architecture tests with ArchUnit

Design rules written on a wiki erode one shortcut at a time. **ArchUnit** (Apache-2.0) turns them into JUnit tests over your compiled classes: a controller that reaches into a repository fails the build, with the exact line that broke the rule.

ArchitectureTest — ArchUnit 1.5.1, on the Spring project — 3 ✔ · 0.354 s

```
/** Design rules, checked on every build like any other test. */
@AnalyzeClasses(packages = "shop",
    importOptions = ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

  @ArchTest
  static final ArchRule CONTROLLERS_DONT_USE_REPOSITORIES =
      noClasses().that().haveSimpleNameEndingWith("Controller")
          .should().dependOnClassesThat()
          .haveSimpleNameEndingWith("Repository")
          .because("controllers go through a service");

  @ArchTest
  static final ArchRule CONTROLLERS_DONT_RETURN_ENTITIES =
      noClasses().that().haveSimpleNameEndingWith("Controller")
          .should().dependOnClassesThat()
          .areAnnotatedWith(Entity.class)
          .because("lazy JPA proxies must not reach JSON");

  @ArchTest
  static final ArchRule SERVICES_DONT_KNOW_THE_WEB =
      noClasses().that().haveSimpleNameEndingWith("Service")
          .should().dependOnClassesThat()
          .resideInAPackage("org.springframework.web..");
}
```

a deliberate violation (test sources only)

```
/** A deliberate violation, used to prove the rule bites. */
class ShortcutController {
  private final OrderRepository repository;

  ShortcutController(OrderRepository repository) {
    this.repository = repository;
  }

  long count() { return repository.count(); }
}
```

proving the rule bites — 1 ✔ · 0.877 s

```
  @Test
  void controllerCallingARepositoryIsReported() {
    var classes = new ClassFileImporter()
        .importPackages("shop.archdemo");

    assertThatThrownBy(() -> ArchitectureTest
        .CONTROLLERS_DONT_USE_REPOSITORIES.check(classes))
        .isInstanceOf(AssertionError.class)
        .hasMessageContaining("was violated (")
        .satisfies(e -> System.out.println(e.getMessage()));
  }
```

real output · the violation ArchUnit reported

```
Architecture Violation [Priority: MEDIUM] - Rule 'no classes that have simple name ending with 'Controller' should depend on classes
that have simple name ending with 'Repository', because controllers go through a service' was violated (3 times):
Constructor <ShortcutController.<init>(OrderRepository)> has parameter of type <OrderRepository> in (ShortcutController.java:0)
Field <ShortcutController.repository> has type <OrderRepository> in (ShortcutController.java:0)
Method <ShortcutController.count()> calls method <OrderRepository.count()> in (ShortcutController.java:13)
```

- **Rules worth having:** layer direction, no cycles between packages (`slices().matching("..shop.(*)..").should().beFreeOfCycles()`), naming, and "no `java.util.Date`" style bans.
- **Add `.because(…)`** — the reason appears in the failure, so the next developer learns the rule instead of fighting it.
- **Prove each rule can fail** once, as above; a rule with a typo in a package name passes forever.
- **Existing violations?** `FreezingArchRule` records today's and fails only on new ones — adopt rules without a big-bang clean-up.

## Rules worth having — ArchUnit API reference

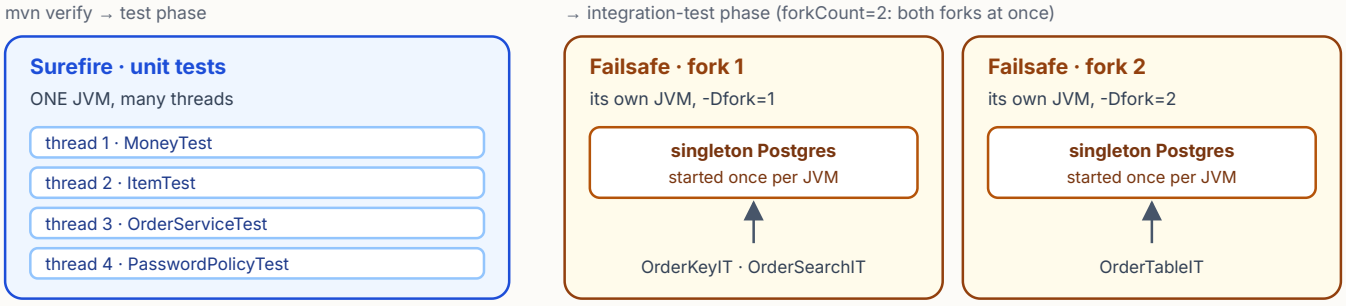
| RULE | ARCHUNIT |
|---|---|
| **Layers only call downwards** | `layeredArchitecture().consideringAllDependencies().layer(…).definedBy(…).whereLayer(…).mayOnlyBeAccessedByLayers(…)` |
| **No cycles between feature packages** | `slices().matching("..shop.(*)..").should().beFreeOfCycles()` |
| **Domain code stays framework-free** | `noClasses().that().resideInAPackage("..domain..").should().dependOnClassesThat().resideInAPackage("org.springframework..")` |
| **No field injection** | `GeneralCodingRules.NO_CLASSES_SHOULD_USE_FIELD_INJECTION` |
| **No `System.out` or generic exceptions** | `NO_CLASSES_SHOULD_ACCESS_STANDARD_STREAMS`, `NO_CLASSES_SHOULD_THROW_GENERIC_EXCEPTIONS` |
| **Naming** | `classes().that().areAnnotatedWith(RestController.class).should().haveSimpleNameEndingWith("Controller")` |

The three rules in `ArchitectureTest` above ran in CI. The rows in this table come from ArchUnit's user guide and its `GeneralCodingRules` library — not run here.

06 · QUALITY & PROCESS

# Parallel tests: threads for unit, JVMs for integration

Unit tests share nothing, so they run as **threads in one JVM**. Integration tests share heavy, stateful resources — a database, a container, a port — so they run in **several JVMs**, each with its own copy of those resources.




| | UNIT TESTS | INTEGRATION TESTS |
|---|---|---|
| **Run side by side as** | threads in one JVM | separate JVM processes (forks) |
| **What tests share** | nothing — objects live in memory | containers, ports, database rows, Spring context, files |
| **Configure with** | JUnit `junit.jupiter.execution.parallel.*` | Failsafe `forkCount`; Gradle `maxParallelForks` |
| **Isolation comes from** | no shared mutable state; `@ResourceLock` for the rest | each fork has its own JVM, singletons and data |
| **Cost of one more** | a thread — almost free | a JVM start and its own containers |
| **Usual setting** | on, for every build | a few forks — measure, don't guess (p.47) |

pom.xml — Surefire threads, Failsafe forks

```xml
<!-- UNIT tests: one JVM, many threads -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.6.0</version>
  <configuration>
    <properties>
      <configurationParameters>
        junit.jupiter.execution.parallel.enabled = true
        junit.jupiter.execution.parallel.mode.default = concurrent
        junit.jupiter.execution.parallel.mode.classes.default = concurrent
      </configurationParameters>
    </properties>
  </configuration>
</plugin>

<!-- INTEGRATION tests: many JVMs, one singleton container each -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.6.0</version>
  <configuration>
    <forkCount>${it.forks}</forkCount>
    <reuseForks>true</reuseForks>
    <!-- -Dfork=1, 2, ...: which fork this JVM is -->
    <argLine>@{argLine} -Xshare:off -Dfork=${surefire.forkNumber}
             -javaagent:${org.mockito:mockito-core:jar}</argLine>
  </configuration>
  ...
</plugin>
```

build.gradle.kts — Gradle forks

```
// INTEGRATION tests: many JVMs, one singleton
container each
val integrationTest = tasks.register<Test>
("integrationTest") {
  description = "Runs *IT classes in parallel JVMs"
  group = "verification"
  testClassesDirs =
sourceSets["integrationTest"].output.classesDirs
  classpath =
sourceSets["integrationTest"].runtimeClasspath
  useJUnitPlatform()
  maxParallelForks = 2
  shouldRunAfter(tasks.test)
  testLogging { showStandardStreams = true }
}
tasks.check { dependsOn(integrationTest) }
```

- **forkCount=2** runs up to two test JVMs at once; `2C` means two per CPU core. **reuseForks=true** (the default) keeps each JVM for many classes, so its singletons start once.
- `${surefire.forkNumber}` becomes 1…*forkCount*. **Our run:** a system property that was *only* the placeholder arrived empty, while `shop_${surefire.forkNumber}` and `argLine` worked. Print it once.
- Surefire's own `parallel` parameter is for JUnit 4 and TestNG; for JUnit Jupiter pass JUnit's settings, as above.
- Gradle: `maxParallelForks` (default 1). Each worker gets a unique `org.gradle.test.worker` property — unique, but *not* 1…N: our runs got workers 2 and 3.
- `reuseForks=false` (Maven) or `forkEvery` (Gradle) recycles JVMs — for leaky tests only; singletons restart too.
- Maven's `-T` (parallel modules) multiplies the forks: 3 modules × `forkCount=2` can mean 6 JVMs and 6 containers.

06 · QUALITY & PROCESS

# Singleton containers: one per test JVM

Testcontainers' documented pattern for sharing a container is the **singleton container**: a `static` field in a base class, started in a static initialiser. It starts once when the first subclass loads, every IT class in that JVM reuses it, and the **Ryuk** container removes it when the JVM exits. With forks: **one container per fork**.

src/it/java/shop/SharedPostgres.java (Testcontainers 2.0.5)

```
abstract class SharedPostgres {
  static final String FORK = System.getProperty("fork",
      System.getProperty("org.gradle.test.worker", "1"));
  static final PostgreSQLContainer DB =
      new PostgreSQLContainer("postgres:17-alpine");

  static {
    DB.start();                 // once per JVM, never stop() here
    // … create the schema, log the container id
  }

  static Connection connect() throws SQLException {
    return DriverManager.getConnection(
        DB.getJdbcUrl(), DB.getUsername(), DB.getPassword());
  }
  // log(), uniqueId(), insert() …
}
```

an IT class just extends it

```
class OrderKeyIT extends SharedPostgres {
  @BeforeAll static void hello() {
    log("OrderKeyIT uses " + DB.getJdbcUrl());
  }

  @Test void duplicateIdIsRejected() throws Exception {
    String id = uniqueId("dup");
    try (var c = connect()) {
      insert(c, id, "10");
      assertThatThrownBy(() -> insert(c, id, "20"))
          .isInstanceOf(SQLException.class)
          .extracting(e -> ((SQLException) e).getSQLState())
          .isEqualTo("23505");          // unique_violation
    }
  }
}
```

**WATCH OUT**

Don't annotate the singleton with `@Container` and don't call `stop()`: the JUnit extension stops a `static` `@Container` after the last test of its class, so the next class would find it gone.

## Real run: Failsafe, `forkCount=2`, Docker on GitHub Actions

Apache Maven 3.10.0 · Failsafe 3.6.0 · 2 containers for 4 database IT classes — 10 tests ✔

```
[fork 1 pid 2665] Postgres 2d7f090afddf on port 32771
[fork 1 pid 2665] OrderTableIT uses port 32771
[fork 2 pid 2666] Postgres f761dd55aa56 on port 32770
[fork 2 pid 2666] OrderKeyIT uses port 32770
  ✔ OrderKeyIT — 6.480 s
[fork 2 pid 2666] OrderSearchIT uses port 32770
  ✔ OrderTableIT — 7.083 s
  ✔ OrderSearchIT — 0.074 s
```

- **One container per JVM.** Fork 1 started a container for `OrderKeyIT` (6.5 s, mostly start-up); `OrderSearchIT` reused it and took 0.07 s. Fork 2 started its own for `OrderTableIT`.
- **Measure it.** Same build, warm Docker, two runs each on a 4-CPU runner: one IT JVM **15.2–15.3 s**, two JVMs **19.1–19.6 s** for the whole `mvn verify`. Two forks were slower here: two containers start at once on a small machine and our tests are short. Forks pay off when IT classes run for minutes.
- **Spring:** the context cache is per JVM too, so each fork builds its own context; wire the container in with `@DynamicPropertySource`.
- Gradle with `maxParallelForks = 2`: the same, one container per worker.

## Four ways to share a database

| STRATEGY | HOW | TRADE-OFF | USE FOR |
|---|---|---|---|
| **Singleton per JVM** | one container per fork, shared by its classes | isolation for free; N forks = N containers | **Default choice** |
| **One shared database** | one DB; schema `shop_${surefire.forkNumber}` per fork | fewer containers; isolate the data yourself | large suites, heavy images |
| **`@Container` per class** | a new container for every test class | simplest, slowest | a few classes only |
| **Reusable containers** | `withReuse(true)` + `testcontainers.reuse.enable=true` | survives across runs; experimental | **local only**, not CI |

06 · QUALITY & PROCESS

# Faster builds: build cache & test shards

When the suite gets slow, don't delete tests — stop re-running what didn't change, and split what must run across machines. Both below are real runs on GitHub Actions.

## 1 · Build cache: skip unchanged work

real output · clean test, without and with the cache

```
### gradle clean test --no-build-cache
> Task :compileJava
> Task :compileTestJava
> Task :testClasses
> Task :test
BUILD SUCCESSFUL in 5s
4 actionable tasks: 4 executed
### gradle clean test --build-cache
> Task :compileJava FROM-CACHE
> Task :compileTestJava FROM-CACHE
> Task :testClasses UP-TO-DATE
> Task :test FROM-CACHE
BUILD SUCCESSFUL in 858ms
4 actionable tasks: 1 executed, 3 from cache
```

- Gradle hashes every task's inputs — sources, dependencies, JVM arguments. With `--build-cache` (or `org.gradle.caching=true`), a task whose inputs were seen before is restored **FROM-CACHE** instead of run — tests included.
- **Real run:** 5s with the cache off, every task executed; 858ms with it on — compiled classes and test results came from the cache that `gradle/actions/setup-gradle` keeps between CI runs.
- Share a **remote cache** between CI and laptops so a branch built once is cheap everywhere.
- Maven has no built-in build cache; the Apache Maven Build Cache Extension adds one.

## 2 · Shards: split test classes across machines

.github/workflows/build.yml — two shards

```yaml
shard:
  runs-on: ubuntu-latest
  strategy:
    matrix: { shard: [1, 2] }       # two machines, half the classes each
  defaults: { run: { shell: 'bash -eo pipefail {0}' } }
  steps:
    - uses: actions/checkout@v7
    - uses: actions/setup-java@v6
      with: { distribution: temurin, java-version: '21' }
    - uses: gradle/actions/setup-gradle@v6
    - name: Run this shard's test classes
      run: |
        mkdir -p logs
        classes=$(cd src/test/java && find . -name '*Test.java' \
          | sed 's#^./##; s#.java$##; s#/#.#g' | sort \
          | awk -v n=2 -v k=${{ matrix.shard }} 'NR % n == k % n')
        echo "shard ${{ matrix.shard }}/2: $(echo $classes | wc -w) classes" | tee logs/shard.log
        ./gradlew --console=plain test $(printf -- '--tests %s ' $classes) 2>&1 | tail -3 | tee -a logs/shard.log
        # … then count the tests this shard ran
```

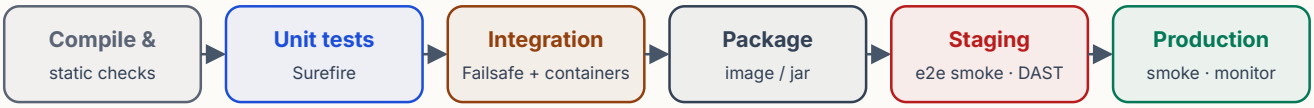
- **Real run:** shard 1 ran 23 tests and shard 2 ran 21 — all 44, on two machines at once.
- Split by class, never by method: `@BeforeAll` setup and shared fixtures stay together.
- Balance by duration, not count, once the suite grows: the slowest shard decides the total time.
- Collect every shard's JUnit XML report in one place, so a failure is visible without opening each job.

## More ways to spend less time

| TECHNIQUE | HOW | WATCH OUT |
|---|---|---|
| **Tags: fast vs slow** | `@Tag("slow")`; Surefire `groups` / `excludedGroups`, Gradle `includeTags` | slow tests still need a home — nightly |
| **Run one module or class** | `mvn -pl orders -am test`; `-Dtest=OrderServiceTest`; Gradle `--tests` | for the inner loop, not for CI gates |
| **Parallel forks** | threads for unit tests, forks for ITs (p.46) | measure first — it was slower for us |
| **Test selection** | run only tests affected by a change (e.g. Develocity Predictive Test Selection, commercial) | keep a full run nightly |

06 · QUALITY & PROCESS

# Build tools & CI pipelines




← every push, minutes · fail fast

nightly: full e2e, performance, mutation, DAST →

## Maven: Surefire and Failsafe

| | SUREFIRE | FAILSAFE |
|---|---|---|
| **Runs in phase** | `test` | `integration-test`, then `verify` fails the build |
| **Default class names** | `Test*`, `*Test`, `*Tests`, `*TestCase` | `IT*`, `*IT`, `*ITCase` |
| **Use for** | Fast unit tests | Tests needing a started app, DB or containers |
| **Command** | `mvn test` | `mvn verify` — so `post-integration-test` clean-up runs |

Both are at version 3.6.0. With Gradle, register a second `Test` task (or use the JVM Test Suite plugin) for integration tests.

## Run tests in parallel

junit-platform.properties

```
junit.jupiter.execution.parallel.enabled=true
junit.jupiter.execution.parallel.mode.default=concurrent
junit.jupiter.execution.parallel.mode.classes.default=concurrent
junit.jupiter.execution.parallel.config.strategy=dynamic
```

Our 44 tests on 4 CPUs, three runs each: sequential **1,979–2,032 ms**, parallel **1,444–1,475 ms**, all passing. Parallel runs only work if tests share no state — a good test of your tests.

## Where should this test live? A Spring Boot service

| WHAT YOU'RE CHECKING | LEVEL | TOOLING |
|---|---|---|
| **Pricing, validation, state rules** | unit | JUnit + AssertJ, real value objects, no Spring |
| **Service orchestration (charge → save → e-mail)** | unit | Mockito for ports: gateway, repository, mailer |
| **URL mapping, JSON shape, validation errors, status codes** | integration | `@WebMvcTest` + `MockMvcTester` + `@MockitoBean` |
| **SQL, mappings, migrations, transactions** | integration | `@DataJpaTest` + Testcontainers (same DB as prod) |
| **Outbound HTTP client: timeouts, errors, parsing** | integration | WireMock or `@RestClientTest` |
| **Agreement with another team's service** | contract | Pact or Spring Cloud Contract |
| **Sign-up, checkout, payment journeys** | end-to-end | Playwright against staging; a few smoke tests in prod |

.github/workflows/build.yml · sketch — not run here

```yaml
name: build
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-java@v6
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: ./mvnw -B verify    # unit + integration
      - uses: actions/upload-artifact@v7
        if: always()             # keep reports on failure
        with: { name: test-reports, path: '**/target/*-reports/*.xml' }
```

## Pipeline habits

- **Fastest first.** Static checks and unit tests run before anything slow, so most failures arrive in minutes.
- **Every pull request** runs the full unit and integration suite; merging is blocked on red.
- **Keep reports** (JUnit XML, coverage, Playwright traces) as build artefacts, especially on failure.
- **Budget the time.** If the main build creeps past ~10 minutes, parallelise, split jobs or push slow suites to nightly.
- **Quarantine flaky tests** in a non-blocking job and fix them — never "retry until green".
- **Show results in the pull request:** test, coverage and mutation summaries next to the diff get read; a link to a dashboard doesn't.

06 · QUALITY & PROCESS

# Team testing conventions — a starting point

A guide lists options; a team needs decisions. These defaults come from the pages of this guide — each row links to the page that explains it. Copy them into your repository's `CONTRIBUTING.md`, then change what doesn't fit your team. The numbers are suggestions, not standards.

| TOPIC | OUR DEFAULT | WHY |
|---|---|---|
| **Naming & layout** | | |
| **Unit test classes** | `OrderServiceTest` in `src/test/java`, same package as the class | p.49 |
| **Integration tests** | `OrderFlowIT` — run by Failsafe in `verify`; may need Docker | p.46 |
| **Test names** | say the behaviour: `unknownOrderIs404`, not `testGet2` | p.04 |
| **Shape** | one behaviour per test; Arrange · Act · Assert with blank lines between | p.04 |
| **Writing tests** | | |
| **Assertions** | AssertJ everywhere; one logical assertion per test, `assertAll`/soft assertions for groups | p.09 |
| **Test doubles** | mock only the boundaries you own (gateway, repository, mailer); real value objects | p.13 |
| **Mockito** | `MockitoExtension` (strict stubs); `BDDMockito` in given/when/then tests | p.15 |
| **Time & async** | inject a `Clock`; no `Thread.sleep` — use Awaitility or latches | p.32 |
| **Test data** | builders with defaults (`anOrder().withItems(2).build()`); unique IDs per test, so parallel runs never collide | p.46 |
| **Spring Boot** | | |
| **Default level** | slices first (`@WebMvcTest`, `@DataJpaTest`); `@SpringBootTest` for a few flows | p.19 |
| **Transactions** | no `@Transactional` on `@SpringBootTest` tests; one committed-path test per repository | p.20 |
| **Database** | Testcontainers with the production database and version, schema from the same Flyway/Liquibase migrations — not H2 as a stand-in | p.19 |
| **Build gates** (suggested numbers — tune them) | | |
| **Speed** | main build under ~10 minutes; slow suites move to nightly | p.49 |
| **Coverage** | branch coverage on *new* code ≥ 80% (JaCoCo `check` or a Sonar gate) | p.42 |
| **Mutation** | PIT on core modules in pull requests, threshold ≥ 80%; everything nightly | p.41 |
| **Duplication** | CPD on test code too (`includeTests`); start at 50 tokens, lower once clean | p.44 |
| **Parallelism** | unit tests in threads always; integration forks only after measuring | p.46 |
| **Flaky tests** | quarantine within a day; fix or delete within the sprint; never retry until green | p.32 |

## Where each check runs

| WHEN | CHECKS | ROLE |
|---|---|---|
| **Every push** | unit tests, CPD, PMD, Checkstyle, SpotBugs | minutes; fail fast |
| **Every pull request** | integration tests, coverage gate on new code, PIT on the changed module | blocks the merge |
| **Nightly** | full PIT, end-to-end and performance suites, Dependency-Check, duplicate-test report | trends and alerts, not gates |

## Four questions for every test in code review

**Would it fail?**
If the behaviour broke, would this test go red? (PIT answers this for you.)

**Right level?**
Is it the lowest level that can catch the bug — unit before integration before end-to-end?

**Readable?**
Does the name state the expected behaviour, and does a failure message say what went wrong?

**Independent?**
No shared state, real time, sleeps or test order; safe to run in parallel.

06 · QUALITY & PROCESS

# Check your understanding

Twenty questions to check your understanding — for self-review, team discussions or onboarding. Each has a short answer to start from; the index points to the page that covers the topic in depth.

### Q01 Unit vs integration test?

A unit test checks one class with its collaborators replaced, in memory and in milliseconds. An integration test checks your code together with real parts — database, HTTP, framework — and is slower but catches wiring and configuration bugs.

### Q02 Mock vs stub?

A stub returns canned answers so the code under test can run; you then check the result (state). A mock is set up with expectations about calls, which you verify (behaviour). Mockito objects can play either role.

### Q03 What does `@InjectMocks` do?

Creates the class under test and injects `@Mock` / `@Spy` fields by constructor, then setter, then field. It fails silently when it can't inject — many teams prefer calling the constructor explicitly.

### Q04 `when().thenReturn()` vs `doReturn().when()`?

Use `doReturn` for spies (the `when` form calls the real method first) and `doThrow` / `doNothing` for void methods.

### Q05 How do you test a private method?

Through the public method that uses it. If that is too hard, the private logic probably wants to be its own class with its own tests.

### Q06 How do you test code that uses the current time?

Inject a `java.time.Clock`; production passes `Clock.systemUTC()`, tests pass `Clock.fixed(…)`.

### Q07 What is a flaky test and how do you fix it?

One that passes and fails without code changes. Find the uncontrolled input — time, randomness, order, shared state, network, sleeps — and control it; quarantine until fixed.

### Q08 Is 100% coverage the goal?

No. Coverage shows what ran, not what was checked. Use it to find untested code; use mutation testing to judge test strength.

### Q09 `@BeforeEach` vs `@BeforeAll`?

`@BeforeEach` runs before every test on a fresh instance; `@BeforeAll` runs once per class and must be static unless `@TestInstance(PER_CLASS)`.

### Q10 Can Mockito mock final classes and statics?

Yes. Since Mockito 5 the inline mock maker is the default, so final classes and methods work; statics need `mockStatic` in a try-with-resources block.

### Q11 What is the test pyramid?

Many fast unit tests, fewer integration tests, very few end-to-end tests — because cost and flakiness grow as tests get broader. The testing trophy puts more weight on integration tests.

### Q12 How do you test a REST controller in Spring Boot?

`@WebMvcTest` with `MockMvcTester` (or `MockMvc`) and `@MockitoBean` services for the web layer; `@SpringBootTest(webEnvironment = RANDOM_PORT)` plus Testcontainers for a full flow.

### Q13 What is TDD and when is it worth it?

Red–green–refactor in short loops. Most valuable for logic with clear rules (pricing, parsing, validation); less so for exploratory UI work.

### Q14 How do you test exceptions in JUnit?

`assertThrows(Type.class, () → …)` returns the exception so you can check its message; AssertJ offers `assertThatThrownBy(…)`. Avoid the JUnit 4 `@Test(expected = …)` style — it passes if *any* line throws.

### Q15 What is mutation testing?

A tool (PIT) makes small changes to your code — flips `⩾` to `>`, removes a call — and re-runs the tests. Surviving mutants show where tests run code without really checking it.

### Q16 How do you make tests run in parallel safely?

Remove shared mutable state: no statics, unique test data, temp dirs, random ports. Then enable `junit.jupiter.execution.parallel.enabled`; use `@ResourceLock` for anything truly shared.

### Q17 Surefire vs Failsafe?

Surefire runs unit tests in the `test` phase (`*Test`). Failsafe runs integration tests (`*IT`) in `integration-test` and fails the build in `verify`, so clean-up still happens.

### Q18 What do you test in a microservice architecture?

Each service with unit and integration tests, the agreements between services with contract tests, and only a few critical journeys end to end.

### Q19 How do you find duplicate or redundant tests?

Run PIT with the full mutation matrix and compare which mutants each test kills; equal sets are merge candidates. Add copy-paste detection (PMD CPD, SonarQube) and review — boundary tests often look redundant but aren't.

### Q20 What is contract testing?

Consumers publish the requests and responses they rely on; the provider's build verifies them (Pact, Spring Cloud Contract). It replaces many slow end-to-end tests between services.

PIN THIS PAGE

# Cheat sheet

*Testing Java at a glance*

## JUNIT 6 ANNOTATIONS

| | |
|---|---|
| `@Test` | a test |
| `@ParameterizedTest` | one run per argument set |
| `@BeforeEach / @AfterEach` | around every test |
| `@BeforeAll / @AfterAll` | once per class (static) |
| `@Nested` | group related tests |
| `@Tag("slow")` | filter what runs |
| `@Disabled("why")` | skip, with a reason |
| `@TempDir` | fresh temp folder |
| `@ExtendWith` | plug in extensions |

## ASSERTIONS

| | |
|---|---|
| `assertEquals(exp, act)` | expected first |
| `assertThrows(X.class, …)` | returns the exception |
| `assertAll(…)` | report every failure |
| `assertThat(x).isEqualTo(y)` | AssertJ |
| `.containsExactly(…)` | order matters |
| `.extracting(Order::total)` | map, then check |
| `assertThatThrownBy(…)` | then .hasMessage(...) |

## MOCKITO

| | |
|---|---|
| `when(m.f()).thenReturn(v)` | stub |
| `doReturn(v).when(spy).f()` | stub a spy |
| `doThrow(e).when(m).voidF()` | stub a void |
| `verify(m).f(eq(1), any())` | all matchers or none |
| `verify(m, never()).f()` | not called |
| `verifyNoInteractions(m)` | untouched |
| `@Captor ArgumentCaptor<T>` | grab the argument |
| `mockStatic(X.class)` | try-with-resources |

## TEST DOUBLES

- **Dummy** — fills a parameter, never used
- **Stub** — canned answers
- **Spy** — stub that records calls
- **Mock** — expectations, verified
- **Fake** — working shortcut (in-memory repo)

## WHICH LEVEL?

- **Unit** — rules, calculations, orchestration
- **Integration** — SQL, HTTP, JSON, config
- **Contract** — agreements between services
- **End-to-end** — a few money-making journeys
- Push each check to the **cheapest level** that catches it

## FLAKY? CONTROL THE INPUT

- Time → inject `Clock`
- Random → seed it
- Sleep → wait for a condition
- Shared state → fresh fixtures, unique IDs
- Network → stub or container

## COMMANDS

| | |
|---|---|
| `mvn test` | unit tests (Surefire) |
| `mvn verify` | + integration tests (Failsafe) |
| `mvn test -Dtest=MoneyTest` | one class |
| `mvn test -Dtest='MoneyTest#rejects*'` | matching methods |
| `mvn test -Dgroups=fast` | only @Tag("fast") |
| `./gradlew test --tests '*MoneyTest'` | Gradle filter |
| `mvn test-compile pitest:mutationCoverage` | mutation testing (PIT) |
| `target/site/jacoco/index.html` | JaCoCo report after mvn verify |

## GOLDEN RULES

- Watch every new test **fail first**
- One behaviour per test, named as a sentence
- Arrange · Act · Assert — and nothing else
- Test behaviour through the **public API**
- Don't mock values or what you don't own
- Boundaries: test on and either side of every edge
- Coverage finds gaps; **PIT** measures strength — merge unit + IT coverage
- A flaky test is a bug — quarantine and fix it

## VERSIONS USED HERE

- JUnit **6.1.3** · Java 17+
- Mockito **5.24.0** · AssertJ **3.27.7**
- Run on OpenJDK **21**

## PERFORMANCE

- Report **p50 / p95 / p99**, not the mean
- Warm up the JVM before measuring
- Little's Law: users = rate × time

## SECURITY IN CI

- **SAST** + **SCA** + secrets on every build
- **DAST** (ZAP) nightly on staging
- Write access-control tests yourself

BEFORE YOU SHIP

# Testing checklist

### Unit tests

- [ ] Business rules tested at every boundary
- [ ] Each test checks one behaviour, named as a sentence
- [ ] No real clock, randomness, network or files

### Doubles

- [ ] Mocks only at the edges (ports you own)
- [ ] No mocked value objects
- [ ] No unused stubs (strict stubs on)

### Integration

- [ ] Real database engine in a container
- [ ] HTTP status, headers and body checked
- [ ] Unhappy paths: 4xx, 5xx, timeouts

### Suite health

- [ ] Runs in random order and in parallel
- [ ] No flaky tests in the main build
- [ ] Main build under ~10 minutes

### Strength

- [ ] Coverage reviewed for untested code
- [ ] Mutation testing on critical modules
- [ ] Every bug fix starts with a failing test

### Beyond

- [ ] A few end-to-end journeys + prod smoke
- [ ] Load test with percentile targets
- [ ] SAST, SCA, DAST and access-control tests

## References & further reading


| | |
|---|---|
| `Beck 2002` | Test-Driven Development: By Example |
| `Fowler 2004/2007` | Mocks Aren't Stubs — martinfowler.com |
| `Dodds 2018/2021` | The Testing Trophy (tweet 2018, article 2021) — kentcdodds.com |
| `DeMillo et al. 1978` | Hints on Test Data Selection — mutation testing |
| `docs.junit.org` | JUnit 6 user guide & release notes |
| `assertj.github.io` | AssertJ core documentation |
| `java.testcontainers.org` | Testcontainers for Java |
| `Meszaros 2007` | xUnit Test Patterns — the test double names |
| `Cohn 2009` | Succeeding with Agile — the test pyramid |
| `Robinson 2006` | Consumer-Driven Contracts — martinfowler.com |
| `Winters et al. 2020` | Software Engineering at Google, ch. 11–14 |
| `site.mockito.org` | Mockito javadoc (incl. agent setup) |
| `docs.spring.io` | Spring Boot 4 testing reference |
| `pitest.org · jacoco.org` | Mutation and coverage tools |


## How this guide was verified

| CRITERION | HOW IT WAS CHECKED | RESULT |
|---|---|---|
| **Code is real** | Unit tests pass on JUnit 6.1.3, Mockito 5.24.0, AssertJ 3.27.7, Java 21. The Maven, Gradle and Spring Boot 4.1 builds — Failsafe forks, PostgreSQL, Redis, Kafka, GreenMail, Moto, Azurite, WireMock, Pact, Flyway, Playwright, mock-oauth2-server, ArchUnit, Spring Batch, jcstress, JaCoCo, PIT, CPD, build cache, shards — run on GitHub Actions, failing cases included. The other examples (PIT, coverage, load, parallel, CPD) are re-run in the same pipeline, and this PDF is rebuilt from those outputs on every push. | **Pass** |
| **Sketches labelled** | The CI-pipeline sketch and the ArchUnit rule reference are marked as not run. LocalStack now needs an account token, so its test was skipped and says so. APIs and versions were checked against official docs, release tags or source. The conventions page states opinions and says so. | **Pass** |
| **Facts & versions** | Every version, date, annotation, module name and default checked against official docs, release tags or source; Mockito strictness, Surefire's fork-number handling and CPD defaults confirmed by running them. Every sentence that interprets an output is re-checked against the fresh output on each build (claims.py). | **Pass** |
| **Layout** | Automated scan of every page on each build: clipped code or an overflowing box fails the build; large empty areas are reported. Pages also reviewed visually. | **Pass** |
| **Navigation** | Clickable contents, page references and footer links, nested bookmarks for every page, A–Z index. | **Pass** |
| **Accessibility** | Tagged PDF, document language, alt text on diagrams and screenshots. Automated WCAG 2.1 AA contrast audit of every text element on each build: 0 failures, or the build fails. | **Pass** |
| **Fonts & text** | All fonts embedded (checked on each build); text, code and ₹ signs search and copy correctly. | **Pass** |

FIND ANYTHING

# Index A–Z


## A

AAA (Arrange · Act · Assert) 04
Acceptance test 03
Access control tests 36
Accessibility testing 34
ApplicationContextRunner 37
ArchUnit 45
argLine 42, 16
ArgumentCaptor 14, 15
assertAll 09
AssertJ 09, 10
assertThrows 09
assertTimeoutPreemptively 32
Assumptions 10
Awaitility 32, 26, 24
Azurite 28

## B

BDD / Gherkin 38
BDDMockito 15
BigDecimal money 09
Boundary value analysis 06, 39
Branch coverage 39
Build cache 48

## C

Cache TTL 22
@Cacheable / @CacheEvict 22
Canary release 37
Chaos engineering 36
Characterization tests 17
Cheat sheet 52
Classicist vs mockist 14
Clock (java.time) 32, 14, 24
Code coverage 39
Composed annotations 11
Consumer-driven contracts 31
Contract testing 31, 51
Coordinated omission 35
CountDownLatch 32
Coverage: unit vs integration 42
@CsvSource 06, 10
Cucumber 38
Cypress 34

## D

DAST 36
Database migrations 21
@DataJpaTest 19, 20
Decision table 06
Dependency-Check (OWASP) 36, 44
@DisplayName 08
doReturn 16, 15
Dummy 13
Duplicate tests 43

## E

E-mail tests 27
@EmbeddedKafka 26
End-to-end tests 34
Equivalence partitioning 06
Equivalent mutant 40
@ExtendWith 08, 10

## F

Failsafe (Maven) 49, 46
Failure messages 09, 16
Fake 13, 18
FastCSV 07
Feature flags 37
@FieldSource 10
FIRST principles 04
Flaky tests 32, 49, 23
Flyway / Liquibase 21
forkCount / reuseForks 46
Four pillars of a good test 05
fullMutationMatrix 43
Fuzzing (Jazzer) 36

## G

Gatling 35
Given / When / Then 04, 15, 38
Golden master / approval 17
Gradle 07, 49
GreenMail 27

## H

Hostile input tests 36
HttpClient (JDK) 18
HttpServer (JDK) 18, 35

## I

Idempotent consumer 26
immediateWrites (Redis cache) 23
@InjectMocks 51
InOrder 15
Integration tests 18

## J

JaCoCo 39
JaCoCo check (gate) 42
JaCoCo merge / report 42
jcstress 33
JMeter 35
JMH 35
JobOperatorTestUtils 25
JSpecify 07
JUnit 4 migration 07
JUnit 6 changes 07, 10
JUnit extensions 11
JUnit Platform 07

## K

k6 35
Kafka 26
Keycloak 29
Kill matrix 43

## L

LazyInitializationException 20
Legacy code 17
Lenient stubs 16
Lifecycle 08
Little's Law 35
Load testing 35
LocalStack 28
Locators 34
Log assertions 12
Lost update (race) 33

## M

Mailpit 27
Maven setup 07
maxParallelForks (Gradle) 46
@MethodSource 10
Micrometer 12
Mock 13, 14
Mockito 14, 15, 16
Mockito as -javaagent 16
@MockitoBean 19
MockitoExtension 14
MockitoSettings 16
MockMvcTester 19
MockOAuth2Server 29
mockStatic 16
Moto (AWS) 28
Mutation score 39, 40
Mutation testing (PIT) 39
mutationThreshold 41
Mutator groups (DEFAULTS, STRONGER) 39, 40, 41

## N

Nagle's algorithm 35
Naming tests 04
@Nested 10
New-code coverage 42

## O

OAuth2 / JWT tests 29
OutputCaptureExtension 12

## P

Pact 19, 31
Pact Broker / can-i-deploy 31
Page Object 34
Parallel execution 49, 46
@ParameterizedTest 06, 10
ParameterResolver 11
Percentiles (p95, p99) 35
pitest-junit5-plugin 40, 41
pitest-maven 41, 39
Playwright 34
PMD CPD 44
Port 0 (free port) 18
prepare-agent-integration 42
Property-based testing 06

## Q

Quality gate 42, 41
quoteTextArguments 10, 07

## R

Record (Java) 08, 16
Redis 22, 23
Redundant tests 43
@RepeatedTest 11
Repeating a flaky test 23
report-aggregate 42
Resource leak tests 40
REST Assured 34
Retries (Resilience4j) 30
Risk-based testing 05

## S

SAST 36
SCA 36
@Scheduled 24
Seams 17
Selenium 34
Self-check questions 51
Self-invocation (proxies) 22
@ServiceConnection 19
Singleton container 47
Skip / filter (Batch) 25
Smoke test 03, 34
SonarQube 42, 44
SpotBugs 44
Spring Batch 25
Spring Boot test slices 19, 49
@SpringBatchTest 25
Spy 13, 16
State vs behaviour verification 13
Strict stubs 16
Stub 13
Surefire (Maven) 49, 07
surefire.forkNumber 46, 47
Surviving mutant 40, 41
Synthetic checks 37

## T

@Tag 10
TCP_NODELAY 35
TDD 38
Team conventions 50
@TempDir 32
Test data builders 50
Test doubles 13
Test pyramid 04
Test sharding 48
Test sizes (Google) 03
Test smells 39
Testcontainers 19, 18, 47
Testing in production 37
Testing trophy 04
@TestInstance 08
@TestMethodOrder 08
thenAnswer 14
TIMED_OUT (PIT) 40, 41
Timeouts 30
Toxiproxy 36
@Transactional in tests 20

## V

verify 14, 15
Virtual threads 35
Visual regression 34

## W

@WebMvcTest 19, 49
What not to test 05
WireMock 19, 30
withHistory (PIT) 41

## Z

ZAP 36


## Three ways to read this guide

**New to testing**

Foundations p.03, first test p.08, assertions p.09, test doubles p.13, Mockito p.14, then the cheat sheet p.52.

**Improving an existing suite**

Team conventions p.50, flaky tests p.32, mutation testing p.40, unit + IT coverage p.42, duplicates p.43, plugins p.44, parallel ITs p.46, CI p.49.

**Quick refresher**

Pyramid p.04, doubles p.13, Mockito pitfalls p.16, Spring slices p.19, TDD p.38, Q&A p.51.